



Politechnika Warszawska

**Wydział Matematyki
i Nauk Informatycznych**

Reprezentacja Wiedzy

Procesy działań złożonych

Część teoretyczna — wersja poprawiona

Zespół projektowy nr 4:

Wojciech Jasiński
Krzysztof Gryboś
Jarosław Bieniek
Filip Filipkowski
Sebastian Rydz
Jakub Pietrzak
Sandra Adamiec

Warszawa, czerwiec 2026

Spis treści

1	Wprowadzenie	2
1.1	Cel projektu	2
1.2	Klasa $\mathbb{DS4}$ – warunki Z1–Z7	2
1.3	Mapa zapożyczeń z wykładów	2
1.4	Ograniczenia $\mathbb{DS4}$	2
2	Język akcji	3
2.1	Sygnatura	3
2.2	Formuły zdaniowe	3
2.3	Stany	3
2.4	Składnia	4
2.4.1	Zdania dziedziny — tabela zbiorcza	4
2.4.2	Pełna BNF dziedziny	4
2.4.3	Opis poszczególnych zdań	5
2.4.4	Dziedzina D	6
2.5	Semantyka akcji prostych	6
2.5.1	Stany dopuszczalne Σ	6
2.5.2	Stany początkowe Σ_0	6
2.5.3	Funkcja Res_0	7
2.5.4	Funkcja New	7
2.5.5	Minimalizacja — $\text{Res}(A, \sigma)$	7
2.5.6	Niewykonalność	8
2.6	Model dziedziny	8
2.6.1	Prawdziwość zdań wartościowych	8
2.6.2	Warunki poprawności modelu (M1–M3)	8
2.6.3	Jednoznaczność modelu	9
3	Język kwerend	9
3.1	Akcja złożona $\mathbb{A}$	9
3.2	Literały i DNF	10
3.3	Konflikty	10
3.4	Dekompozycje $\Delta(\mathbb{A}, \sigma)$	11
3.5	$\text{Res}(\mathbb{A}, \sigma)$ przez dekompozycje	12
3.6	Uwaga o „restricted inheritance”	13
3.7	Porównanie z treścią projektu	13
3.8	Proces $\mathbb{P}$	14
3.9	Ślad wykonania Ψ_S	14
3.10	Zdania języka kwerend	14
3.11	Składnia kwerend	14
3.12	Semantyka formalna	16
4	Przykłady	17
4.1	YSP z rosyjską ruletką	17
4.2	Producent i konsumenci	18
4.3	Dwa przełączniki, żarówka i alarm	19
4.4	Dwóch robotników wynoszących stół	22
5	Podsumowanie	25

1 Wprowadzenie

1.1 Cel projektu

Zadanie polega na zaprojektowaniu dwóch formalnych języków dla klasy $\mathbb{DS4}$ systemów dynamicznych:

- *języka akcji* — służącego do opisu dziedziny (fluenty, akcje, ich skutki, ograniczenia),
- *języka kwerend* — pozwalającego zadawać pytania o wykonywalność procesów i osiągalność celów.

Klasa $\mathbb{DS4}$ łączy cechy kilku języków akcji znanych z wykładów. Mechanizmy inercji, niedeterminizmu, ramifikacji i niewykonalności pochodzą z języka AR (wykład 2). Obsługę akcji złożonych projektujemy na podstawie języka AC (wykład 5) — adaptujemy mechanizm detekcji konfliktów i dekompozycji do warunków $\mathbb{DS4}$, w których opisy dziedzin zawierają wyłącznie akcje proste.

1.2 Klasa $\mathbb{DS4}$ – warunki Z1–Z7

Systemy klasy $\mathbb{DS4}$ są zdefiniowane przez siedem fundamentalnych założeń, które określają ich semantykę i możliwości modelowania:

1. **Z1. Prawo inercji** — fluenty, które nie są bezpośrednio lub pośrednio zmieniane przez akcję, zachowują swoją wartość w kolejnym stanie (np. jeśli akcja „malowanie ściany” zmienia tylko kolor, to położenie drzwi pozostaje bez zmian).
2. **Z2. Pełna informacja o strukturze** — system zakłada dostęp do kompletnej wiedzy o zbiorze fluentów F , akcji A oraz wszystkich ich potencjalnych skutkach zdefiniowanych w modelu.
3. **Z3. Niedeterminizm działań** — wykonanie pojedynczej akcji w konkretnym stanie może prowadzić do wielu alternatywnych stanów wynikowych (np. rzut monetą skutkuje orłem lub reszką).
4. **Z4. Warunki wstępne i wynikowe** — każda akcja posiada zdefiniowaną formułę logiczną, która musi być spełniona przed jej wykonaniem; w przypadku niespełnienia warunku wstępnego akcja nie wywołuje żadnych zmian (efekt pusty).
5. **Z5. Niewykonalność akcji** — model dopuszcza istnienie stanów, w których dana akcja jest całkowicie zablokowana i nie może zostać podjęta przez agenta.
6. **Z6. Warunki integralności (ramifikacje)** — globalne więzy logiczne nakładane na stany wpływają na skutki uboczne akcji; zmiana jednego fluentu może wymusić zmianę innego, aby zachować spójność systemu.
7. **Z7. Opis częściowy** — język pozwala na specyfikację stanów, w których wartości niektórych fluentów są nieznane, co dotyczy zarówno stanu początkowego, jak i stanów osiągniętych po sekwencjach akcji.

1.3 Mapa zapożyczeń z wykładów

1.4 Ograniczenia $\mathbb{DS4}$

Klasa $\mathbb{DS4}$ wprowadza trzy świadome uproszczenia względem pełnego języka AC : (i) opis dziedziny dopuszcza wyłącznie akcje proste, a akcje złożone pojawiają się dopiero w języku kwerend; (ii) nie ma jawnej klauzuli **impossible** dla zbiorów akcji — o wykonalności zbioru rozstrzyga

Tabela 1: Mapowanie mechanizmów klasy $\mathbb{DS4}$ na materiały teoretyczne.

Mechanizm klasy $\mathbb{DS4}$	Źródłowy język	Wykład / Materiały
Inercja, ramifikacje, niedeterminizm	AR	Wykład 2
Niewykonalność akcji	AR	Wykład 2
Semantyka kwerend (<i>necessary/possibly</i>)	AR	Wykład 3
Konflikty i dekompozycja akcji złożonych	AC	Wykład 5
Opisy częściowe i niepełna informacja	AR	Wykład 2, 7

mechanizm konfliktów i dekompozycji (§3.3); (iii) nie stosujemy dziedziczenia restrykcyjnego (*restricted inheritance*) — każda akcja złożona jest dekomponowana do akcji prostych. Uproszczenia te nie zmniejszają siły modelowania: każdą niedozwoloną kombinację akcji można wyrazić przez ograniczenia integralności (Z6) lub warunki wstępne (Z4).

2 Język akcji

Język akcji służy do formalnego opisu dziedziny: deklaruje fluenty, akcje proste oraz ich skutki, warunki niewykonalności i ograniczenia integralności. W tej sekcji podajemy kolejno jego sygnaturę i formuły, składnię zdań, semantykę akcji prostych oraz model dziedziny, którym język ten się domyka. Zgodnie z założeniami klasy $\mathbb{DS4}$ na poziomie opisu dziedziny występują wyłącznie akcje proste.

2.1 Sygnatura

Formalną podstawą opisu jest sygnatura $\Upsilon = (\mathcal{F}, \mathcal{A})$, gdzie:

- $\mathcal{F}$ — skończony zbiór **fluentów** (zmiennych zdaniowych) opisujących stan świata,
- $\mathcal{A}$ — skończony zbiór **akcji prostych**.

W klasie $\mathbb{DS4}$ zbiór $\mathcal{A}$ zawiera wyłącznie akcje proste; akcje złożone nie należą do sygnatury i powstają dopiero w kwerendach.

2.2 Formuły zdaniowe

Język $\mathcal{L}(\Upsilon)$ pozwala budować złożone wyrażenia (warunki wstępne, skutki, więzy) za pomocą formuł zdaniowych. Gramatyka formuł α nad Υ ma postać

$$\alpha ::= f \mid \neg\alpha \mid \alpha \wedge \beta \mid \alpha \vee \beta \mid \alpha \rightarrow \beta \mid \alpha \leftrightarrow \beta \mid \top \mid \perp,$$

gdzie $f \in \mathcal{F}$, a symbole $\top$ oraz $\perp$ oznaczają odpowiednio prawdę i fałsz.

2.3 Stany

Stanem σ nazywamy funkcję wartościowania (totalną), która każdemu fluentowi ze zbioru $\mathcal{F}$ przypisuje dokładnie jedną wartość logiczną: $\sigma: \mathcal{F} \rightarrow \{0, 1\}$. Relację spełnialności formuły α w stanie σ , oznaczaną jako $\sigma \models \alpha$, definiujemy rekurencyjnie zgodnie z klasyczną logiką zdaniową:

- $\sigma \models f$ wtw, gdy $\sigma(f) = 1$,
- $\sigma \models \neg\alpha$ wtw, gdy $\sigma \not\models \alpha$,
- $\sigma \models \alpha \wedge \beta$ wtw, gdy $\sigma \models \alpha$ oraz $\sigma \models \beta$,
- $\sigma \models \alpha \vee \beta$ wtw, gdy $\sigma \models \alpha$ lub $\sigma \models \beta$,

- $\sigma \models \alpha \rightarrow \beta$ wtw, gdy $\sigma \not\models \alpha$ lub $\sigma \models \beta$,
- $\sigma \models \alpha \leftrightarrow \beta$ wtw, gdy $\sigma \models \alpha$ jest równoważne $\sigma \models \beta$,
- $\sigma \models \top$ zawsze, a $\sigma \models \perp$ nigdy.

2.4 Składnia

Projektowany język akcji służy do opisu dziedzin należących do klasy $\mathbb{DS4}$ systemów dynamicznych. Zgodnie z treścią zadania, w opisach dziedzin występują wyłącznie akcje proste, tj. elementy ustalonego zbioru akcji $\mathcal{A}$. Akcje złożone, rozumiane jako zbiory akcji prostych wykonywanych współbieżnie, nie należą zatem do składni języka opisu dziedziny, lecz pojawiają się dopiero na poziomie języka kwerend.

Przypomnijmy oznaczenia z §2: α, β, π oznaczają dowolne formuły zdaniowe nad $\mathcal{F}$, f — fluent, a A — akcję prostą ze zbioru $\mathcal{A}$.

2.4.1 Zdania dziedziny — tabela zbiorcza

W projektowanym języku akcji wyróżniamy *sześć* podstawowych typów zdań, zebranych w tabeli 2. Cztery z nich (**causes**, **releases**, **always**, **noninertial**) opisują lokalną dynamikę lub strukturę dziedziny, a dwa pozostałe (**after** oraz **observable after**) pełnią rolę zdań specyfikacyjnych, odnoszących się do własności zachodzących po wykonaniu ciągu akcji prostych. Oprócz nich używamy dwóch *skrótów* składniowych — **impossible** oraz **initially** — które nie są osobnymi typami zdań, lecz szczególnymi przypadkami konstrukcji **causes** i **after** (tabela 3).

Tabela 2: Podstawowe typy zdań języka akcji dla klasy $\mathbb{DS4}$

Zdanie (zapis formalny)	Znaczenie intuicyjne	Źródło
$A \text{ causes } \alpha \text{ if } \pi$	Wykonanie akcji prostej A w stanie spełniającym π wymusza zajście efektu α .	AR
$A \text{ releases } f \text{ if } \pi$	Wykonanie akcji A w stanie spełniającym π zwalnia fluent f spod działania prawa inercji.	AR
always α	Formuła α musi być prawdziwa w każdym dopuszczalnym stanie dziedziny.	AR
noninertial f	Fluent f nie podlega prawu inercji.	AR
$\alpha \text{ after } A_1, \dots, A_n$	Po wykonaniu ciągu akcji prostych $A_1, \dots, A_n$ formuła α zachodzi zawsze.	AR
observable $\alpha \text{ after } A_1, \dots, A_n$	Po wykonaniu ciągu akcji prostych $A_1, \dots, A_n$ formuła α może zachodzić.	AR

Dla wygody zapisu wprowadzamy dwa *skrót*y (tabela 3). Nie są to osobne typy zdań: każdy z nich jest szczególnym przypadkiem jednej z powyższych konstrukcji i w semantyce traktujemy go wyłącznie przez jego rozwinięcie.

W każdym z powyższych schematów zakładamy, że $A, A_1, \dots, A_n \in \mathcal{A}$, a zatem wszystkie występujące akcje są akcjami prostymi. Jest to zgodne z warunkami klasy $\mathbb{DS4}$ i odróżnia poziom opisu dziedziny od poziomu języka kwerend, na którym dopiero dopuszczane są akcje złożone.

2.4.2 Pełna BNF dziedziny

Składnię języka akcji definiujemy następująco. Przez α, β oraz π oznaczamy dowolne formuły zdaniowe nad zbiorem fluentów $\mathcal{F}$, zaś przez A dowolną akcję prostą należącą do $\mathcal{A}$.

Tabela 3: Skróty składniowe i ich rozwinięcia

Skrót	Rozwinięcie	Znaczenie
impossible A if π	A causes $\perp$ if π	Akcja A jest niewykonalna w każdym stanie spełniającym π — żaden stan wynikowy nie może spełnić $\perp$.
initially α	α after ε	Wiedza o stanie początkowym; zdanie after dla pustego ciągu akcji ε .

$$\begin{aligned}
\langle \text{dziedzina} \rangle &::= \langle \text{zdanie} \rangle \mid \langle \text{zdanie} \rangle ; \langle \text{dziedzina} \rangle \\
\langle \text{zdanie} \rangle &::= \langle \text{efekt} \rangle \mid \langle \text{zwolnienie} \rangle \mid \langle \text{niewykonalnosc} \rangle \mid \langle \text{ograniczenie} \rangle \\
&\quad \mid \langle \text{nieinercyjnosc} \rangle \mid \langle \text{początkowe} \rangle \mid \langle \text{wartosciowe} \rangle \mid \langle \text{obserwacyjne} \rangle \\
\langle \text{efekt} \rangle &::= A \text{ **causes** } \alpha \text{ if } \pi \\
\langle \text{zwolnienie} \rangle &::= A \text{ **releases** } f \text{ if } \pi \\
\langle \text{niewykonalnosc} \rangle &::= \text{impossible } A \text{ if } \pi \\
\langle \text{ograniczenie} \rangle &::= \text{always } \alpha \\
\langle \text{nieinercyjnosc} \rangle &::= \text{noninertial } f \\
\langle \text{początkowe} \rangle &::= \text{initially } \alpha \\
\langle \text{wartosciowe} \rangle &::= \alpha \text{ **after** } A_1, \dots, A_n \\
\langle \text{obserwacyjne} \rangle &::= \text{observable } \alpha \text{ **after** } A_1, \dots, A_n
\end{aligned}$$

Powyższa gramatyka obejmuje wszystkie typy zdań dopuszczalnych w języku akcji dla klasy $\mathbb{DS4}$; produkcje $\langle \text{niewykonalnosc} \rangle$ oraz $\langle \text{początkowe} \rangle$ to omówione wyżej skróty **impossible** i **initially** (tabela 3). Składnia nie zawiera reguł dla akcji złożonych.

2.4.3 Opis poszczególnych zdań

Poniżej omawiamy zwięźle te zdania, których sens wykracza poza tabelę 2.

A **causes** α if π — reguła efektowa: jeśli A jest wykonana w stanie spełniającym π , to każdy dopuszczalny stan wynikowy spełnia α . To podstawowy mechanizm reprezentacji pewnych skutków działań.

A **releases** f if π — zwolnienie spod inercji: po wykonaniu A w stanie spełniającym π wartość fluentu f może, lecz nie musi, zostać zachowana. Służy do modelowania niedeterminizmu lokalnego.

always α — ograniczenie integralności: α musi zachodzić w każdym dopuszczalnym stanie. Ograniczenia te, wymuszając spójność stanów wynikowych, realizują mechanizm ramifikacji.

noninertial f — fluent f nie podlega inercji, więc jego wartość nie musi być zachowywana między stanami (przydatne dla fluentów pochodnych).

α **after** $A_1, \dots, A_n$ — zdanie specyfikacyjne wyrażające własność *konieczną*: po wykonaniu ciągu $A_1, \dots, A_n$ formuła α zachodzi zawsze. Jest warunkiem na model całej sekwencji, nie lokalną regułą przejścia.

observable α **after** $A_1, \dots, A_n$ — jego specyfikacyjny odpowiednik dla własności *możliwej*: istnieje co najmniej jeden przebieg ciągu $A_1, \dots, A_n$ kończący się w stanie spełniającym α .

Skróty. **impossible** A **if** π **to** A **causes** $\perp$ **if** π : w stanie spełniającym π żaden stan wynikowy nie spełnia $\perp$, więc A jest tam niewykonalna (§2.5); dotyczy wyłącznie akcji prostych. **initially** α to α **after** ε (pusty ciąg akcji): opisuje stan początkowy i wyznacza zbiór Σ_0 . Dopuszczalny jest opis częściowy — zbiór takich zdań może pozostawiać niektóre fluenty nieustalone.

2.4.4 Dziedzina D

Przez dziedzinę działania rozumiemy skończony zbiór zdań języka akcji nad ustaloną sygnaturą $\Upsilon = (\mathcal{F}, \mathcal{A})$; oznaczamy ją przez D . Zbiór D jest pełną specyfikacją systemu na poziomie akcji prostych — względem niego definiujemy dalej semantykę.

2.5 Semantyka akcji prostych

W tej sekcji definiujemy semantykę języka akcji na poziomie akcji prostych, względem ustalonej dziedziny D nad sygnaturą $\Upsilon = (\mathcal{F}, \mathcal{A})$. Korzystamy z oznaczeń wprowadzonych w §2: stan to wartościowanie $\sigma : \mathcal{F} \rightarrow \{0, 1\}$, a zapis $\sigma \models \alpha$ oznacza prawdziwość formuły α w stanie σ .

2.5.1 Stany dopuszczalne Σ

Nie każdy stan jest stanem dopuszczalnym z punktu widzenia opisu dziedziny. Zdania postaci **always** narzucają globalne ograniczenia integralności, które muszą być spełnione w każdym stanie dopuszczalnym. Zbiór stanów dopuszczalnych definiujemy więc następująco:

$$\Sigma = \{\sigma : \text{dla każdego } (\mathbf{always} \alpha) \in D, \sigma \models \alpha\}.$$

Zbiór Σ zawiera dokładnie te stany, które spełniają wszystkie ograniczenia integralności występujące w dziedzinie. Jeżeli pewien stan narusza choć jedno zdanie typu **always**, to nie należy do Σ i nie może wystąpić ani jako stan początkowy, ani jako wynik wykonania jakiegokolwiek akcji. W ten sposób ograniczenia integralności wyznaczają przestrzeń stanów, pomiędzy którymi mogą zachodzić przejścia. Mechanizm ten odpowiada za reprezentację ramifikacji: stan wynikowy musi być nie tylko zgodny z bezpośrednimi efektami działania, lecz również z globalną strukturą dziedziny.

2.5.2 Stany początkowe Σ_0

Zbiór stanów początkowych definiujemy jako podzbiór zbioru stanów dopuszczalnych spełniających wszystkie zdania początkowe:

$$\Sigma_0 = \{\sigma \in \Sigma : \text{dla każdego } (\mathbf{initially} \alpha) \in D, \sigma \models \alpha\}.$$

Tak określony zbiór może być jednoelementowy albo wieloelementowy. W szczególności, jeżeli opis początkowy nie ustala wartości wszystkich fluentów, to kilka różnych stanów może być zgodnych z wiedzą początkową. Przyjęcie zbioru Σ_0 , zamiast pojedynczego wyróżnionego stanu początkowego, pozwala modelować częściowy opis stanu początkowego bez wprowadzania dodatkowych konstrukcji składniowych. Uwzględniamy jedynie te stany z Σ_0 , które są zgodne ze zdaniem wartościowymi i obserwacyjnymi występującymi w dziedzinie, tj. ze zdaniem postaci

$$\alpha \text{ after } A_1, \dots, A_n \quad \text{oraz} \quad \mathbf{observable} \alpha \text{ after } A_1, \dots, A_n.$$

2.5.3 Funkcja Res_0

Niech $A \in \mathcal{A}$ będzie akcją prostą, a $\sigma \in \Sigma$ stanem dopuszczalnym. Pierwszym krokiem przy wyznaczaniu wyników wykonania akcji A w stanie σ jest określenie wszystkich stanów dopuszczalnych zgodnych z bezpośrednimi efektami aktywnych reguł efektowych. W tym celu definiujemy funkcję pomocniczą $\text{Res}_0 : \mathcal{A} \times \Sigma \rightarrow 2^\Sigma$ daną wzorem

$$\text{Res}_0(A, \sigma) = \{\sigma' \in \Sigma : \text{dla każdego } (A \textbf{ causes } \alpha \textbf{ if } \pi) \in D, \sigma \models \pi \Rightarrow \sigma' \models \alpha\}.$$

Zbiór $\text{Res}_0(A, \sigma)$ zawiera zatem wszystkie stany dopuszczalne, które spełniają efekty wynikające z wykonania akcji A w stanie σ . Definicja ta uwzględnia zarówno skutki bezpośrednie, jak i ograniczenia integralności, ponieważ kandydackie stany wynikowe są wybierane wyłącznie spośród elementów zbioru Σ . Funkcja Res_0 nie rozstrzyga jednak jeszcze, które z tych stanów należy uznać za rzeczywiste wyniki wykonania akcji, ponieważ na tym etapie nie jest jeszcze uwzględniona zasada inercji.

2.5.4 Funkcja New

W celu sformalizowania zasady minimalnej zmiany wprowadzamy funkcję

$$\text{New} : \mathcal{A} \times \Sigma \times \Sigma \rightarrow 2^{\text{Lit}}, \quad \text{Lit} = \{f, \neg f : f \in \mathcal{F}\},$$

która dla akcji prostej A , stanu wejściowego σ oraz kandydata na stan wynikowy σ' wyznacza zbiór $\text{New}(A, \sigma, \sigma')$ literalów nowych względem stanu wejściowego; Lit oznacza zbiór wszystkich literalów nad $\mathcal{F}$. Intuicyjnie zbiór ten opisuje zakres zmiany wymaganej do przejścia od σ do σ' .

Formalnie:

$$\text{New}(A, \sigma, \sigma') = \{l : \sigma' \models l \text{ oraz zachodzi co najmniej jeden z warunków (1)–(2)}\},$$

gdzie $\mathcal{F}_I = \mathcal{F} \setminus \{f : (\textbf{noninertial } f) \in D\}$ jest zbiorem fluentów inercyjnych (tj. niezadeklarowanych jako **noninertial**), a warunki (1)–(2) są następujące:

1. $l \in \{f, \neg f\}$ dla pewnego fluentu inercyjnego $f \in \mathcal{F}_I$, oraz $\sigma(f) \neq \sigma'(f)$;
2. $l \in \{f, \neg f\}$ oraz istnieje reguła

$$(A \textbf{ releases } f \textbf{ if } \pi) \in D$$

taka, że

$$\sigma \models \pi.$$

Pierwszy warunek odpowiada rzeczywistej zmianie wartości fluentu inercyjnego. Drugi uwzględnia przypadek, w którym aktywna reguła **releases** zwalnia wskazany fluent spod działania inercji. Warto zauważyć, że fluenty zadeklarowane jako **noninertial** nie są traktowane jako zwykłe fluenty inercyjne, a zatem ich zmiana nie zwiększa kosztu przejścia w sensie zasady minimalnej zmiany.

2.5.5 Minimalizacja — $\text{Res}(A, \sigma)$

Właściwy zbiór wyników wykonania akcji prostej A w stanie σ otrzymujemy przez wybór tych stanów z $\text{Res}_0(A, \sigma)$, dla których zbiór $\text{New}(A, \sigma, \sigma')$ jest minimalny względem inkluzji. Definiujemy zatem funkcję wyniku $\text{Res} : \mathcal{A} \times \Sigma \rightarrow 2^\Sigma$ wzorem

$$\text{Res}(A, \sigma) = \{\sigma' \in \text{Res}_0(A, \sigma) : \nexists \sigma'' \in \text{Res}_0(A, \sigma) (\text{New}(A, \sigma, \sigma'') \subset \text{New}(A, \sigma, \sigma'))\}.$$

Zbiór $\text{Res}(A, \sigma)$ zawiera dokładnie te stany, które spełniają wszystkie aktywne efekty bezpośrednie i jednocześnie wprowadzają najmniejszą konieczną zmianę względem stanu wejściowego.

Minimalność rozumiana jest tutaj względem inkluzji zbiorów, a nie względem ich liczności. Dzięki temu zasada inercji przyjmuje postać preferencji dla tych stanów wynikowych, które zachowują jak najwięcej własności stanu poprzedniego, o ile nie stoi to w sprzeczności z efektami akcji ani z ograniczeniami integralności.

Jeżeli kilka stanów osiąga ten sam stopień minimalności, wszystkie pozostają dopuszczalnymi wynikami wykonania akcji. Niedeterminizm może więc wynikać zarówno z obecności reguł **releases**, jak i z wielości różnych minimalnych stanów zgodnych z ograniczeniami dziedziny.

Przykład. Niech $\mathcal{F} = \{p, q\}$ i niech dziedzina zawiera tylko regułę A **causes** p (brak zdań **always**). W stanie $\sigma = \{\neg p, \neg q\}$ kandydatami z $\text{Res}_0(A, \sigma)$ są oba stany spełniające p : $\{p, \neg q\}$ oraz $\{p, q\}$. Pierwszy zmienia jedynie p ($\text{New} = \{p\}$), drugi zmienia dodatkowo q ($\text{New} = \{p, q\}$). Minimalny względem inkluzji jest pierwszy, więc $\text{Res}(A, \sigma) = \{\{p, \neg q\}\}$ — inercja zachowuje wartość q .

2.5.6 Niewykonalność

Osobnego traktowania wymaga przypadek niewykonalności akcji. Jeżeli istnieje reguła

$$(\text{impossible } A \text{ if } \pi) \in D$$

taka, że

$$\sigma \models \pi,$$

to przyjmujemy

$$\text{Res}(A, \sigma) = \emptyset.$$

Oznacza to, że akcja A nie ma w stanie σ żadnego dopuszczalnego wyniku. Reguła **impossible** ma więc charakter priorytetowy — jest to szczególny przypadek reguły efektowej prowadzącej do sprzeczności (A **causes** $\perp$).

2.6 Model dziedziny

Semantykę akcji prostej domykamy pojęciem modelu dziedziny — struktury, która wiąże stany dopuszczalne, stany początkowe oraz funkcję wyniku w jeden obiekt i rozstrzyga prawdziwość zdań specyfikacyjnych dziedziny.

2.6.1 Prawdziwość zdań wartościowych

Posługujemy się tu pojęciem *śladu wykonania* Ψ_S — częściowej funkcji zwracającej stan osiągnięty po wykonaniu ciągu akcji (pełna definicja w §3.8); ciąg akcji prostych $A_1, \dots, A_n$ traktujemy jako proces singletonów. Zdania wartościowe są prawdziwe w $S = (\Sigma, \Sigma_0, \text{Res})$, jeśli zachodzą warunki:

α after $A_1, A_2, \dots, A_n$

Dla każdego $\sigma_0 \in \Sigma_0$ i dla każdej funkcji Ψ_S , jeżeli $\Psi_S((A_1, A_2, \dots, A_n), \sigma_0)$ jest zdefiniowane, to $\Psi_S((A_1, A_2, \dots, A_n), \sigma_0) \models \alpha$.

observable α after $A_1, A_2, \dots, A_n$

Istnieje $\sigma_0 \in \Sigma_0$ oraz funkcja Ψ_S taka, że $\Psi_S((A_1, A_2, \dots, A_n), \sigma_0)$ jest zdefiniowane oraz $\Psi_S((A_1, A_2, \dots, A_n), \sigma_0) \models \alpha$.

2.6.2 Warunki poprawności modelu (M1–M3)

Struktura $S = (\Sigma, \Sigma_0, \text{Res})$ jest modelem dziedziny D , jeśli:

(M1) Σ jest zbiorem wszystkich stanów w D , tj. zbiorem wszystkich stanów dopuszczalnych (§2.5.1).

- (M2) Każde zdanie wartościowe α **after** $A_1, \dots, A_n$ oraz obserwacyjne **observable** α **after** $A_1, \dots, A_n$ z D jest prawdziwe w S (§2.6.1); zdania **initially** wyznaczają zbiór Σ_0 (§2.5.2).
- (M3) Dla każdej akcji prostej $A \in \mathcal{A}$ i każdego stanu $\sigma \in \Sigma$ zbiór $\text{Res}(A, \sigma) \subseteq \text{Res}_0(A, \sigma)$ jest zbiorem tych stanów $\sigma' \in \text{Res}_0(A, \sigma)$, dla których $\text{New}(A, \sigma, \sigma')$ jest minimalny względem zawierania (§2.5.5). Rozszerzenie funkcji wyniku na akcje złożone podajemy w §3.5.

Warunek **M1** wymusza spełnienie ograniczeń integralności **always**, warunek **M2** — zgodność ze zdaniami wartościowymi i obserwacyjnymi, a warunek **M3** — zasadę inercji (fluenty niewspomniane w akcji zachowują swoją wartość).

Definicja modelu odpowiada językowi AC.

2.6.3 Jednoznaczność modelu

Jeżeli dziedzina D jest niesprzeczna, to spełniający powyższe warunki model S jest jedyny. Wynika to z faktu, że wszystkie trzy elementy modelu są wyznaczane jednoznacznie. Dziedzina sprzeczna (np. gdy zdania **after/observable** przeczą funkcji Res) nie ma żadnego modelu; relacja $D \approx Q$ jest wtedy spełniona pusto dla każdej kwerendy.

Każda składowa modelu jest wyznaczona jednoznacznie przez swój typ zdań: Σ przez zdania **always**, Σ_0 przez **initially** (z przycięciem zdaniami **after/observable**), a Res przez **causes**, **releases** i **impossible**. Niedeterminizm nie narusza jednoznaczności: $\text{Res}(A, \sigma)$ to zawsze *cały* zbiór możliwych stanów docelowych.

Konsekwencją jednoznaczności modelu jest to, że kwantyfikator po modelach występujący w definicji relacji $D \approx Q$ (§3.12) sprowadza się do sprawdzenia w tym jednym modelu.

3 Język kwerend

W niniejszym rozdziale rozszerzamy semantykę wprowadzoną w §2.5 na poziom akcji złożonych, rozumianych jako zbiory akcji prostych wykonywanych równocześnie. Ponieważ w klasie $\mathbb{DS4}$ zdania dziedziny odnoszą się wyłącznie do akcji prostych, akcje złożone pojawiają się dopiero na poziomie procesów i są konstruowane dynamicznie. Podstawę teoretyczną stanowi język AC; poniższe definicje adaptują mechanizmy detekcji konfliktów i dekompozycji znane z tego języka do warunków zadania.

3.1 Akcja złożona $\mathbb{A}$

Akcją złożoną nad sygnaturą $\Upsilon = (\mathcal{F}, \mathcal{A})$ nazywamy dowolny niepusty zbiór akcji prostych

$$\mathbb{A} = \{A_1, \dots, A_k\} \subseteq \mathcal{A}, \quad \mathbb{A} \neq \emptyset.$$

Intuicyjnie, składowe akcji złożonej mają zostać wykonane równocześnie w tym samym stanie. Zbiór wszystkich akcji złożonych oznaczamy przez

$$\mathcal{A}_z = 2^{\mathcal{A}} \setminus \{\emptyset\}.$$

Zbiór ten stanowi alfabet procesów w języku kwerend (§3.8), gdzie każdy krok procesu jest elementem $\mathcal{A}_z$.

Dla wygody notacyjnej przyjmujemy konwencję, zgodnie z którą pojedyncza akcja prosta $A \in \mathcal{A}$ występująca w procesie jest skrótem oznaczającym singleton $\{A\} \in \mathcal{A}_z$. Dzięki temu semantyka akcji złożonych, wprowadzona poniżej, obejmuje jako przypadek szczególny semantykę akcji prostej z §2.5.5: dla $\mathbb{A} = \{A\}$ konstrukcje z niniejszej sekcji redukują się do tych z §2.5.

Podkreślimy, że w klasie $\mathbb{DS4}$ akcje złożone nie są elementami sygnatury ani składni opisu dziedziny. Oznacza to w szczególności, że dla akcji złożonej nie można zadeklarować reguły

causes, **releases** ani **impossible**; jej zachowanie jest w pełni wyznaczone przez zachowanie składowych akcji prostych oraz przez mechanizm dekompozycji opisany dalej.

3.2 Literały i DNF

W analizie skutków akcji i wykrywaniu konfliktów kluczowa jest postać normalna formuł.

- **Literał** to fluent f lub jego negacja $\neg f$.
- **Dysjunkcyjna postać normalna** (DNF) to alternatywa koniunkcji literałów; każdą taką koniunkcję nazywamy *składnikiem* i utożsamiamy ze zbiorem jej literałów.

Każdą formułę można sprowadzić do równoważnej postaci DNF standardowymi przekształceniami (eliminacja $\rightarrow, \leftrightarrow$; prawa De Morgana; dystrybucja koniunkcji względem alternatywy); szczegóły proceduralne należą do dokumentacji technicznej.

Przykład. Formuła $(\neg p \rightarrow q) \wedge r$ ma postać DNF $(p \wedge r) \vee (q \wedge r)$, czyli dwa składniki $C_1 = \{p, r\}$ oraz $C_2 = \{q, r\}$. Rozkład na składniki służy do sprawdzania komplementarności literałów przy akcjach złożonych (§3.3).

3.3 Konflikty

Przy równoczesnym wykonaniu kilku akcji prostych może dojść do sytuacji, w której ich skutki są ze sobą niezgodne — na przykład jedna akcja wymusza zajście literału f , a druga literału $\neg f$. W takim przypadku akcje te nazywamy *kolidującymi* w danym stanie. Definicję konfliktu przyjmujemy za językiem AC i formułujemy ją zawsze względem ustalonego stanu $\sigma \in \Sigma$, ponieważ zestaw aktywnych reguł **causes** oraz **releases** zależy od tego, które warunki wstępne π są w σ spełnione.

Dla danej akcji prostej $A \in \mathcal{A}$ i stanu $\sigma \in \Sigma$ wprowadzamy zbiór *aktywnych efektów* oraz zbiór *aktywnych zwolnień*, dane funkcjami $E : \mathcal{A} \times \Sigma \rightarrow 2^{\mathcal{L}(\mathcal{T})}$ (zbiór formuł efektowych) oraz $R : \mathcal{A} \times \Sigma \rightarrow 2^{\mathcal{F}}$ (zbiór zwalnianych fluentów):

$$E(A, \sigma) = \{\alpha : (A \text{ causes } \alpha \text{ if } \pi) \in D, \sigma \models \pi\},$$

$$R(A, \sigma) = \{f : (A \text{ releases } f \text{ if } \pi) \in D, \sigma \models \pi\}.$$

Niech $A, B \in \mathcal{A}$ będą dwiema różnymi akcjami prostymi, a $\sigma \in \Sigma$ ustalonym stanem dopuszczalnym. Mówimy, że akcje A i B **kolidują w stanie** σ , jeżeli zachodzi co najmniej jeden z dwóch warunków:

Przypadek 1 (causes vs. causes). Istnieją formuły $\alpha \in E(A, \sigma)$ oraz $\beta \in E(B, \sigma)$ takie, że w postaciach normalnych

$$\text{DNF}(\alpha) = C_1^\alpha \vee \dots \vee C_p^\alpha, \quad \text{DNF}(\beta) = C_1^\beta \vee \dots \vee C_q^\beta$$

występuje para składników C_i^α, C_j^β , w której pewien literał z C_i^α jest komplementarny względem pewnego literału z C_j^β , tj. istnieje fluent $f \in \mathcal{F}$ taki, że $f \in C_i^\alpha$ i $\neg f \in C_j^\beta$, albo $\neg f \in C_i^\alpha$ i $f \in C_j^\beta$.

Przypadek 2 (causes vs. releases). Istnieje formuła $\alpha \in E(A, \sigma)$ oraz fluent $f \in R(B, \sigma)$ takie, że f lub $\neg f$ występuje jako literał w pewnym składniku postaci $\text{DNF}(\alpha)$. Symetrycznie dopuszczamy tę sytuację po zamianie ról A i B , tj. $\alpha \in E(B, \sigma)$ oraz $f \in R(A, \sigma)$.

Intuicyjnie, Przypadek 1 opisuje sytuację, w której obie akcje wymuszają bezpośrednio sprzeczne wartości tego samego fluentu, natomiast Przypadek 2 odnosi się do konfliktu między akcją wymuszającą wartość fluentu i akcją, która ten sam fluent jednocześnie zwalnia spod działania inercji. Posłużenie się postacią DNF jest tutaj istotne: formuły efektowe mogą być dowolnie

złożonymi formułami zdaniowymi, a badanie zgodności literalów bezpośrednio w postaci DNF pozwala jednolicie rozpatrywać wszystkie alternatywy sposobu realizacji danego efektu.

Przykład. Jeżeli w stanie σ aktywne są reguły A **causes** f oraz B **causes** $\neg f$, to literały f i $\neg f$ są komplementarne i akcje A, B kolidują (Przypadek 1). Gdyby zamiast B **causes** $\neg f$ aktywna była B **releases** f , konflikt zachodziłby z Przypadku 2 — A wymusza wartość fluentu, który B jednocześnie zwalnia spod inercji.

Z symetrii definicji wynika, że relacja konfliktu w stanie σ jest symetryczna. Możemy zatem zakodować ją jako nieskierowany graf konfliktu

$$G_\sigma(\mathbb{A}) = (V, K), \quad V = \mathbb{A}, \quad K = \{\{A_i, A_j\} : A_i, A_j \in \mathbb{A}, A_i \neq A_j, A_i \text{ i } A_j \text{ kolidują w } \sigma\}.$$

Graf ten wykorzystamy w §3.4 do charakteryzacji dekompozycji.

3.4 Dekompozycje $\Delta(\mathbb{A}, \sigma)$

Jeżeli pewne składowe akcji złożonej $\mathbb{A}$ kolidują w stanie σ , nie możemy wykonać wszystkich z nich równocześnie. Zamiast całkowicie blokować taką akcję, za językiem AC dopuszczamy wykonanie dowolnego maksymalnego podzbioru składowych, który można bezpiecznie uruchomić razem. Taki podzbiór nazywamy dekompozycją.

Najpierw wyróżnimy akcje, które w ogóle mogą zostać wykonane pojedynczo w rozważanym stanie. Mówimy, że akcja prosta $A \in \mathcal{A}$ jest **indywidualnie wykonalna** w stanie σ wtedy i tylko wtedy, gdy $\text{Res}(A, \sigma) \neq \emptyset$. W szczególności każda akcja, dla której w stanie σ aktywne jest zdanie **impossible**, nie jest indywidually wykonalna (por. §2.5). Zbiór składowych indywidually wykonalnych oznaczamy przez

$$V_{\text{exec}}(\mathbb{A}, \sigma) = \{A \in \mathbb{A} : \text{Res}(A, \sigma) \neq \emptyset\}.$$

Definicja (dekompozycja). Podzbiór $\delta \subseteq \mathbb{A}$ jest **dekompozycją** akcji złożonej $\mathbb{A}$ w stanie σ , jeżeli spełnia trzy warunki:

- (D1) **Lokalna wykonalność.** Każda akcja $A \in \delta$ jest indywidually wykonalna w σ , tj. $\delta \subseteq V_{\text{exec}}(\mathbb{A}, \sigma)$.
- (D2) **Bezkonfliktowość.** Żadna para różnych akcji $A, B \in \delta$ nie koliduje w σ . Równoważnie, δ jest zbiorem niezależnym w grafie konfliktu $G_\sigma(\mathbb{A})$.
- (D3) **Maksymalność.** Nie istnieje akcja $A \in V_{\text{exec}}(\mathbb{A}, \sigma) \setminus \delta$, dla której $\delta \cup \{A\}$ nadal spełnia (D1) i (D2).

Zbiór wszystkich dekompozycji akcji $\mathbb{A}$ w stanie σ oznaczamy przez $\Delta(\mathbb{A}, \sigma)$; daje to funkcję $\Delta : \mathcal{A}_z \times \Sigma \rightarrow 2^{2^{\mathcal{A}}}$, której wartością jest rodzina podzbiorów $\mathbb{A}$. Maksymalność rozumiana jest względem inkluzji, a nie względem liczności, co oznacza, że zbiór $\Delta(\mathbb{A}, \sigma)$ może zawierać dekompozycje o różnej liczbie elementów.

Przykład. Niech $\mathbb{A} = \{A, B, C\}$, przy czym wszystkie trzy są indywidually wykonalne w σ , a jedyną kolidującą parą jest $\{A, B\}$. Maksymalnymi zbiorami bezkonfliktowymi są wtedy $\{A, C\}$ oraz $\{B, C\}$, więc $\Delta(\mathbb{A}, \sigma) = \{\{A, C\}, \{B, C\}\}$ — akcji C nie trzeba odrzucać, a wybór między A i B jest źródłem niedeterminizmu.

Lemat (istnienie dekompozycji). Jeżeli $V_{\text{exec}}(\mathbb{A}, \sigma) \neq \emptyset$, to $\Delta(\mathbb{A}, \sigma) \neq \emptyset$.

Szkic dowodu. Niech $A_0 \in V_{\text{exec}}(\mathbb{A}, \sigma)$. Zbiór jednoelementowy $\delta_0 = \{A_0\}$ trywialnie spełnia warunek (D1) oraz, z uwagi na brak par różnych akcji, również warunek (D2). Konstruujemy ciąg zbiorów $\delta_0 \subseteq \delta_1 \subseteq \dots$ następująco: dopóki istnieje akcja $A \in V_{\text{exec}}(\mathbb{A}, \sigma) \setminus \delta_i$ taka, że $\delta_i \cup \{A\}$ wciąż spełnia (D1) oraz (D2), kładziemy $\delta_{i+1} = \delta_i \cup \{A\}$. Ponieważ $\mathbb{A}$ jest zbiorem skończonym, procedura zatrzymuje się po skończonej liczbie kroków na pewnym zbiorze δ^* . Z konstrukcji δ^*

spełnia (D1) i (D2), a jego maksymalność wynika z warunku zatrzymania — żadnej akcji nie można już dodać bez naruszenia (D1) lub (D2). Zatem $\delta^* \in \Delta(\mathbb{A}, \sigma)$. $\square$

Przypadek graniczny. Jeżeli natomiast $V_{\text{exec}}(\mathbb{A}, \sigma) = \emptyset$, tj. żadna składowa akcji $\mathbb{A}$ nie jest w stanie σ indywidualnie wykonalna, to nie istnieje żadna dekompozycja spełniająca (D1), a więc

$$\Delta(\mathbb{A}, \sigma) = \emptyset \quad \text{oraz} \quad \text{Res}(\mathbb{A}, \sigma) = \emptyset.$$

Konwencję tę traktujemy jako integralną część definicji.

3.5 $\text{Res}(\mathbb{A}, \sigma)$ przez dekompozycje

Mając wyznaczony zbiór dekompozycji $\Delta(\mathbb{A}, \sigma)$, definiujemy wynik wykonania akcji złożonej w trzech etapach: najpierw lokalnie, dla pojedynczej dekompozycji δ , analogicznie jak dla akcji prostej z §2.5, a następnie przez sumowanie wyników po wszystkich dekompozycjach.

Efekty i zwolnienia dekompozycji. Dla dekompozycji $\delta \in \Delta(\mathbb{A}, \sigma)$ definiujemy zbiorcze zestawy aktywnych efektów oraz aktywnych zwolnień; tym samym przenosimy oznaczenia E, R (a dalej także New i Res_0) z pojedynczych akcji na dekompozycje i akcje złożone:

$$E(\delta, \sigma) = \bigcup_{A \in \delta} E(A, \sigma), \quad R(\delta, \sigma) = \bigcup_{A \in \delta} R(A, \sigma).$$

Ponieważ w δ nie występują pary kolidujące, suma $E(\delta, \sigma)$ jest (jako zbiór formuł do jednoczesnego spełnienia) niesprzeczna — to właśnie gwarantuje bezkonfliktowość składowych.

Kandydaci i zbiór zmian. Definiujemy zbiór stanów dopuszczalnych zgodnych z efektami dekompozycji:

$$\text{Res}_0(\delta, \sigma) = \{\sigma' \in \Sigma : \text{dla każdego } \alpha \in E(\delta, \sigma), \sigma' \models \alpha\}.$$

Dla każdego kandydata $\sigma' \in \text{Res}_0(\delta, \sigma)$ wyznaczamy zbiór literalów nowych, analogicznie jak w §2.5:

$$\text{New}(\delta, \sigma, \sigma') = \{l : \sigma' \models l \text{ oraz zachodzi co najmniej jeden z warunków (1)–(2)}\},$$

gdzie (1) $l \in \{f, \neg f\}$ dla pewnego fluentu inercyjnego $f \in \mathcal{F}_I$, oraz $\sigma(f) \neq \sigma'(f)$; (2) istnieje fluent $f \in R(\delta, \sigma)$ taki, że $l \in \{f, \neg f\}$. Tak jak dla akcji prostej, fluenty zadeklarowane jako **noninertial** nie są zaliczane do zbioru zmian.

Minimalizacja lokalna. Wynikiem dekompozycji δ w stanie σ jest podzbiór kandydatów minimalnych względem zawierania zbioru New :

$$\text{Res}(\delta, \sigma) = \{\sigma' \in \text{Res}_0(\delta, \sigma) : \nexists \sigma'' \in \text{Res}_0(\delta, \sigma) (\text{New}(\delta, \sigma, \sigma'') \subset \text{New}(\delta, \sigma, \sigma'))\}.$$

Wynik akcji złożonej. W ten sposób rozszerzamy funkcję wyniku na akcje złożone, $\text{Res} : (\mathcal{A} \cup \mathcal{A}_z) \times \Sigma \rightarrow 2^\Sigma$ (dla singletonów $\mathbb{A} = \{A\}$ pokrywa się ona z funkcją z §2.5). Ostateczny zbiór stanów wynikowych akcji złożonej $\mathbb{A}$ w stanie σ powstaje jako suma wyników po wszystkich dekompozycjach:

$$\text{Res}(\mathbb{A}, \sigma) = \bigcup_{\delta \in \Delta(\mathbb{A}, \sigma)} \text{Res}(\delta, \sigma). \quad (1)$$

Analogicznie rozszerzamy funkcję pomocniczą Res_0 na poziom akcji złożonej:

$$\text{Res}_0(\mathbb{A}, \sigma) = \bigcup_{\delta \in \Delta(\mathbb{A}, \sigma)} \text{Res}_0(\delta, \sigma).$$

Suma (1) stanowi operacyjną definicję wyniku akcji złożonej; minimalność zmiany rozstrzygana jest *lokalnie*, w obrębie każdej dekompozycji (§2.5.5). Odpowiada to analogowi warunku (M3) z §2.6.2: stany wynikowe akcji złożonej to dokładnie te stany dopuszczalne, które spełniają łączne efekty pewnej dekompozycji i wprowadzają najmniejszą konieczną zmianę. Jest to świadoma decyzja projektowa: minimalizujemy zmianę per-dekompozycja, a nie globalnie po wszystkich dekompozycjach, jak w oryginalnym warunku (M3) języka *AC*.

Niedeterminizm. Istnienie wielu dekompozycji w tym samym stanie stanowi nowe, niezależne źródło niedeterminizmu w klasie $\mathbb{DS4}$ — obok niedeterminizmu pochodzącego od reguł **releases** i od wielości stanów minimalnych. W przypadku granicznym, gdy żadna para składowych nie koliduje, istnieje dokładnie jedna dekompozycja $\delta = V_{\text{exec}}(\mathbb{A}, \sigma)$, a konstrukcja (1) redukuje się do obliczenia pojedynczego wyniku analogicznego do akcji prostej.

3.6 Uwaga o „restricted inheritance”

W pełnym języku *AC* dziedzina może zawierać zdania o akcjach złożonych (np. $\mathbb{A} \text{ causes } \alpha \text{ if } \pi$); efekt ogłoszony dla $\mathbb{A}$ nie musi być wtedy dziedziczony przez wszystkie jej podzbiory — mechanizm znany jako *restricted inheritance*. W $\mathbb{DS4}$ zdania dotyczą wyłącznie akcji prostych (§2.4), więc problem ten nie występuje: efekty każdej składowej A_i są w całości dziedziczone przez każdą dekompozycję zawierającą A_i (§3.5). Jest to świadome uproszczenie mechanizmu *AC*.

3.7 Porównanie z treścią projektu

Treść zadania projektowego proponuje prostszy mechanizm obsługi współbieżności: akcje złożone wykonują się poprawnie, jeżeli ich składowe operują na rozłącznych zbiorach fluentów. Poniżej pokazujemy, że nasz mechanizm detekcji konfliktów zaczerpnięty z *AC* jest formalnie silniejszy — warunek rozłączności jest jego szczególnym przypadkiem. Tutaj ograniczamy się do argumentu formalnego.

Dla akcji prostej $A \in \mathcal{A}$ oraz stanu $\sigma \in \Sigma$ definiujemy funkcję $\text{Vars} : \mathcal{A} \times \Sigma \rightarrow 2^{\mathcal{F}}$, której wartością jest zbiór fluentów, na które A *potencjalnie wpływa* w stanie σ :

$$\text{Vars}(A, \sigma) = \left(\bigcup_{\alpha \in E(A, \sigma)} \text{fl}(\alpha) \right) \cup R(A, \sigma),$$

gdzie $\text{fl}(\alpha)$ oznacza zbiór fluentów występujących w formule α (niezależnie od polaryzacji literalu). Zbiór $\text{Vars}(A, \sigma)$ obejmuje zatem wszystkie fluenty, które mogą zmienić swoją wartość pod wpływem aktywnych reguł **causes** lub **releases** dla akcji A .

Twierdzenie (rozłączność implikuje brak konfliktu). Niech $A, B \in \mathcal{A}$ będą różnymi akcjami prostymi i niech $\sigma \in \Sigma$ będzie dowolnym stanem dopuszczalnym. Jeżeli $\text{Vars}(A, \sigma) \cap \text{Vars}(B, \sigma) = \emptyset$, to akcje A i B nie kolidują w σ w sensie §3.3.

Dowód. Załóżmy nie wprost, że akcje A i B kolidują w σ . Rozważmy oba przypadki definicji konfliktu.

Przypadek 1. Istnieją $\alpha \in E(A, \sigma)$, $\beta \in E(B, \sigma)$ oraz fluent f taki, że f lub $\neg f$ występuje w pewnym składniku $\text{DNF}(\alpha)$, a komplementarny literal — w pewnym składniku $\text{DNF}(\beta)$. W szczególności fluent f pojawia się w formule α oraz w formule β , zatem $f \in \text{fl}(\alpha) \subseteq \text{Vars}(A, \sigma)$ oraz $f \in \text{fl}(\beta) \subseteq \text{Vars}(B, \sigma)$. Stąd $f \in \text{Vars}(A, \sigma) \cap \text{Vars}(B, \sigma)$ — sprzeczność z założeniem rozłączności.

Przypadek 2. Istnieje (bez utraty ogólności) formuła $\alpha \in E(A, \sigma)$ oraz fluent $f \in R(B, \sigma)$ taki, że f lub $\neg f$ występuje w pewnym składniku $\text{DNF}(\alpha)$. Wówczas $f \in \text{fl}(\alpha) \subseteq \text{Vars}(A, \sigma)$ oraz $f \in R(B, \sigma) \subseteq \text{Vars}(B, \sigma)$, co daje $f \in \text{Vars}(A, \sigma) \cap \text{Vars}(B, \sigma)$ — ponownie sprzeczność. W obu przypadkach otrzymaliśmy sprzeczność, więc akcje A i B nie kolidują w σ . $\square$

Wniosek. Jeżeli składowe akcji złożonej $\mathbb{A}$ operują na parami rozłącznych zbiorach fluentów, to graf konfliktu nie ma krawędzi i $\Delta(\mathbb{A}, \sigma) = \{V_{\text{exec}}(\mathbb{A}, \sigma)\}$ — mechanizm AC i warunek rozłączności dają wtedy ten sam wynik. Implikacja odwrotna nie zachodzi: dwie akcje mogą operować na wspólnym fluencie, a mimo to pozostać bezkonfliktowe, o ile nie prowadzą do komplementarnych literalów (Przypadek 1) ani do pary **causes/releases** na tym fluencie (Przypadek 2). Mechanizm dekompozycji z przykładu producenta i konsumentów (§4.2) ilustruje wynikającą stąd większą swobodę współbieżności niż w oryginalnym AC.

3.8 Proces $\mathbb{P}$

Proces $\mathbb{P} = (\mathbb{A}_1, \dots, \mathbb{A}_n)$ to ciąg akcji złożonych wykonywanych jedna po drugiej; każdy krok $\mathbb{A}_i$ reprezentuje działania równoległe. Zbiór wszystkich procesów oznaczamy przez $\mathcal{P} = \mathcal{A}_z^*$. Kwerendy dotyczą procesów niepustych ($n > 0$); pusty ciąg ε wykorzystujemy jedynie jako bazę rekurencji funkcji Ψ (poniżej) oraz w rozwinięciu skrótu **initially**.

3.9 Ślad wykonania Ψ_S

Pojedyncze wykonanie procesu jest niedeterministyczne: na każdym kroku akcja złożona może prowadzić do wielu stanów wynikowych. *Śladem* wykonania nazywamy ustalony wybór jednego następnika na każdym kroku; ślad reprezentujemy jako funkcję częściową

$$\Psi_S : \mathcal{P} \times \Sigma \rightharpoonup \Sigma,$$

indeksowaną wyborem S , która procesowi $\mathbb{P}$ i stanowi startowemu σ przypisuje stan osiągnięty po wykonaniu $\mathbb{P}$ wzdłuż tego śladu. Ślad jest *dopuszczalny*, jeżeli spełnia warunki rekurencyjne po długości procesu:

$$\Psi_S(\varepsilon, \sigma) = \sigma,$$

a dla procesu niepustego $\mathbb{P} = (\mathbb{A}_1, \dots, \mathbb{A}_k)$ — o ile stan pośredni $\sigma' = \Psi_S((\mathbb{A}_1, \dots, \mathbb{A}_{k-1}), \sigma)$ jest określony —

$$\Psi_S((\mathbb{A}_1, \dots, \mathbb{A}_k), \sigma) \in \text{Res}(\mathbb{A}_k, \sigma').$$

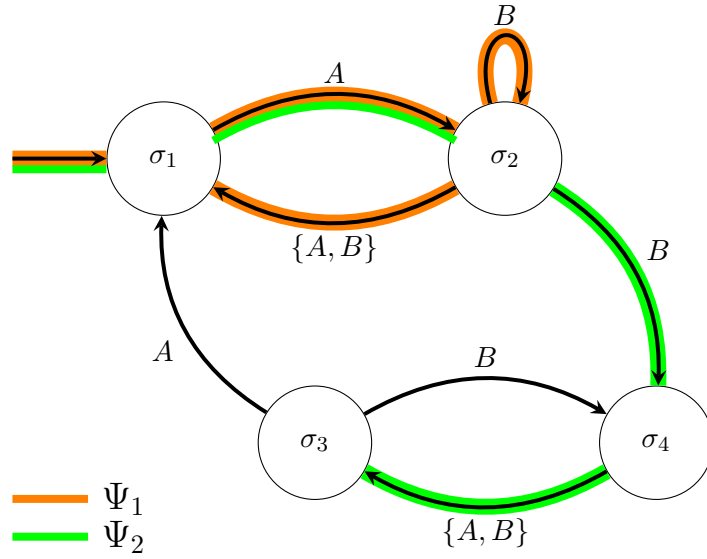
Jeżeli σ' jest nieokreślony albo $\text{Res}(\mathbb{A}_k, \sigma') = \emptyset$ (krok niewykonalny), to $\Psi_S(\mathbb{P}, \sigma)$ pozostaje nieokreślone. Każdy dopuszczalny ślad odpowiada jednej ścieżce w drzewie wykonań procesu, wraz ze stanami pośrednimi; różne rozstrzygnięcia niedeterminizmu dają różne ślady Ψ_S , po których kwantyfikujemy w semantyce kwerend (§3.12). Obrazuje to rysunek 1.

3.10 Zdania języka kwerend

Język kwerend zawiera cztery zapytania (tabela 4): dwa o osiągnięciu stanu po wykonaniu procesu (odpowiadają pytaniu Q2 z treści zadania — czy proces prowadzi do celu) i dwa o wykonalności procesu (pytanie Q1), każde w wersji *necessary* albo *possibly*. Semantykę obu wersji — w szczególności rolę wielu stanów początkowych — precyzujemy w §3.12.

3.11 Składnia kwerend

W języku kwerend występują procesy, będące ciągiem akcji złożonych. Składnię dla tych struktur definiujemy następująco:



Rysunek 1: Przykładowe funkcje Ψ dla procesu $\mathbb{P} = (A, B, \{A, B\})$

Tabela 4: Typy zdań języka kwerend dla klasy $\mathbb{DS4}$

Zdanie (zapis formalny)	Znaczenie intuicyjne
necessary α after $\mathbb{P}$	Czy po wykonaniu procesu $\mathbb{P}$ z każdego stanu początkowego, na każdym śladzie wykonania, formuła α zachodzi?
possibly α after $\mathbb{P}$	Czy dla każdego stanu początkowego istnieje ślad wykonania procesu $\mathbb{P}$, po którym formuła α zachodzi?
necessary executable $\mathbb{P}$	Czy wykonanie procesu $\mathbb{P}$ jest możliwe z każdego stanu początkowego na każdym śladzie wykonania?
possibly executable $\mathbb{P}$	Czy z każdego stanu początkowego istnieje ślad wykonania, na którym proces $\mathbb{P}$ jest wykonalny?

$$\begin{aligned}
\langle \text{proces} \rangle &::= "(" \langle \text{proces-wnetrze} \rangle ")" \\
\langle \text{proces-wnetrze} \rangle &::= \langle \text{proces-wnetrze} \rangle ", " \langle \text{akcja-zlozona} \rangle | \langle \text{akcja-zlozona} \rangle \\
\langle \text{akcja-zlozona} \rangle &::= "{" \langle \text{akcja-zlozona-wnetrze} \rangle "}" \\
\langle \text{akcja-zlozona-wnetrze} \rangle &::= \langle \text{akcja-zlozona-wnetrze} \rangle ", " \langle \text{akcja} \rangle | \langle \text{akcja} \rangle
\end{aligned}$$

Właściwą składnię języka kwerend definiujemy następująco.

$$\begin{aligned}
\langle kw\text{-wartosciowa-nec} \rangle &::= \text{"necessary"} \langle formula \rangle \text{"after"} \langle proces \rangle \\
\langle kw\text{-wartosciowa-pos} \rangle &::= \text{"possibly"} \langle formula \rangle \text{"after"} \langle proces \rangle \\
\langle kw\text{-wykonalnosci-nec} \rangle &::= \text{"necessary executable"} \langle proces \rangle \\
\langle kw\text{-wykonalnosci-pos} \rangle &::= \text{"possibly executable"} \langle proces \rangle
\end{aligned}$$

3.12 Semantyka formalna

Semantykę kwerend opieramy na modelu dziedziny z §2.6. Najpierw wprowadzamy relację konsekwencji $D \approx Q$, następnie wyjaśniamy rolę wielu stanów początkowych, a na końcu podajemy warunki spełnialności dla czterech typów kwerend.

Relacja $D \approx Q$

Kwerenda Q jest konsekwencją dziedziny D , gdy zachodzi w każdym jej modelu S . Ponieważ model dziedziny jest jedyny (§2.6.3), wystarczy sprawdzić Q w tym jednym modelu.

Kwerendy dla wielu stanów początkowych

Z założeń problemu model może mieć wiele różnych stanów początkowych $\sigma_0 \in \Sigma_0$, a z powodu niedeterminizmu Res każdy taki stan może generować wiele różnych śladów wykonania Ψ_S . Kwerendę **necessary** uznajemy za prawdziwą, gdy warunek kwerendy jest spełniony dla *każdego* stanu początkowego oraz *każdego* śladu wykonania. Kwerendę **possibly** uznajemy za prawdziwą, gdy dla *każdego* stanu początkowego istnieje *pewien* ślad wykonania, dla którego warunek kwerendy jest spełniony. Kwantyfikator po Σ_0 jest więc w obu przypadkach uniwersalny: przy opisie częściowym (Z7) rzeczywisty stan początkowy jest nieznany, a zatem poprawna odpowiedź musi uwzględniać każdy stan zgodny z wiedzą początkową.

Definicja spełnialności kwerend

$Q : \text{necessary } \alpha \text{ after } \mathbb{P}$

$D \approx Q$ wtw, gdy w modelu $S = (\Sigma, \Sigma_0, \text{Res})$ dla każdego stanu początkowego $\sigma_0 \in \Sigma_0$ i dla każdej funkcji Ψ_S wartość $\Psi_S(\mathbb{P}, \sigma_0)$ jest zdefiniowana oraz zachodzi $\Psi_S(\mathbb{P}, \sigma_0) \models \alpha$.

$Q : \text{possibly } \alpha \text{ after } \mathbb{P}$

$D \approx Q$ wtw, gdy w modelu $S = (\Sigma, \Sigma_0, \text{Res})$ dla każdego stanu początkowego $\sigma_0 \in \Sigma_0$ istnieje ślad Ψ_S taki, że $\Psi_S(\mathbb{P}, \sigma_0)$ jest zdefiniowane oraz zachodzi $\Psi_S(\mathbb{P}, \sigma_0) \models \alpha$.

$Q : \text{necessary executable } \mathbb{P}$

$D \approx Q$ wtw, gdy w modelu $S = (\Sigma, \Sigma_0, \text{Res})$ dla każdego stanu początkowego $\sigma_0 \in \Sigma_0$ i dla każdej funkcji Ψ_S wartość $\Psi_S(\mathbb{P}, \sigma_0)$ jest zdefiniowana.

$Q : \text{possibly executable } \mathbb{P}$

$D \approx Q$ wtw, gdy w modelu $S = (\Sigma, \Sigma_0, \text{Res})$ dla każdego stanu początkowego $\sigma_0 \in \Sigma_0$ istnieje ślad Ψ_S taki, że $\Psi_S(\mathbb{P}, \sigma_0)$ jest zdefiniowane.

Niewykonalność a kwerendy wartościowe. Jeżeli z pewnego $\sigma_0 \in \Sigma_0$ proces $\mathbb{P}$ nie jest wykonalny — tzn. $\Psi_S(\mathbb{P}, \sigma_0)$ jest nieokreślone dla każdego śladu — to z powyższych definicji **necessary α after $\mathbb{P}$** jest *falszywa*: nie można twierdzić, że α zachodzi po wykonaniu procesu, który się nie kończy. W szczególności **necessary α after $\mathbb{P}$** pociąga **necessary executable**

$\mathbb{P}$. Jest to świadomy wybór mocniejszy od klasycznej formy implikacyjnej z języka *AR* (gdzie na ścieżkach nieokreślonych zdanie jest spełnione pusto).

4 Przykłady

4.1 YSP z rosyjską ruletką

Scenariusz łączący Yale Shooting Problem („rosyjski indyk”) z elementem rosyjskiej ruletki (akcja *Spin*); ilustruje niedeterminizm (**releases**), efekt pusty oraz rolę zdania **impossible**.

Sygnatura:

$$\begin{aligned}\mathcal{F} &= \{loaded, alive\}, \\ \mathcal{A} &= \{Load, Shoot, Spin\}.\end{aligned}$$

Dziedzina D :

initially <i>alive</i>	
initially $\neg loaded$	(broń początkowo rozładowana)
<i>Load</i> causes <i>loaded</i>	
impossible <i>Load</i> if <i>loaded</i>	(nie można ładować naładowanej)
<i>Shoot</i> causes $\neg loaded$	
<i>Shoot</i> causes $\neg alive$ if <i>loaded</i>	
<i>Spin</i> releases <i>loaded</i>	(kręcenie bębenkiem — niedeterminizm)
$\neg loaded$ after <i>Shoot</i>	(specyfikacja: po strzale broń rozładowana)

Stany dopuszczalne (brak **always**, wszystkie 4 stany dopuszczalne):

$$\sigma_0 = \{loaded, alive\}, \quad \sigma_1 = \{loaded, \neg alive\}, \quad \sigma_2 = \{\neg loaded, alive\}, \quad \sigma_3 = \{\neg loaded, \neg alive\}$$

Stany początkowe. Zdania **initially** *alive* oraz **initially** $\neg loaded$ wybierają z Σ jeden stan: $\sigma_2 = \{\neg loaded, alive\}$. Zdanie wartościowe $\neg loaded$ **after** *Shoot* jest dla niego spełnione: $\text{Res}(Shoot, \sigma_2) = \{\sigma_2\} \models \neg loaded$ (broń już rozładowana — efekt pusty). Zatem $\Sigma_0 = \{\sigma_2\}$.

Proces $\mathbb{P} = (\{Load\}, \{Spin\}, \{Shoot\})$.

Wykonanie z $\sigma_2 \in \Sigma_0$:

1. *Load* w σ_2 : efekt *loaded* $\Rightarrow \sigma_0 = \{loaded, alive\}$.
2. *Spin* w σ_0 : fluent *loaded* jest zwolniony (**releases**). Oba stany σ_0 i σ_2 zmieniają się minimalnie, więc $\text{Res}(Spin, \sigma_0) = \{\sigma_0, \sigma_2\}$ (niedeterminizm).
3. *Shoot* po każdej gałęzi:
 - ze σ_0 (broń naładowana): efekty $\neg loaded \wedge \neg alive \Rightarrow \sigma_3$.
 - ze σ_2 (broń pusta): efekt $\neg loaded$ już spełniony, warunek **if** *loaded* fałszywy $\Rightarrow$ efekt pusty, σ_2 bez zmian.

Stany końcowe: $\{\sigma_2, \sigma_3\}$.

Zauważmy, że gdyby broń była naładowana, akcja *Load* byłaby niewykonalna (**impossible** *Load* **if** *loaded*); zdanie **initially** $\neg loaded$ gwarantuje, że proces ma start.

Kwerendy (przypomnijmy: $\Sigma_0 = \{\sigma_2\}$, więc kwantyfikator po stanach początkowych dotyczy jednego stanu):

- **necessary executable** $\mathbb{P}$ — z σ_2 każdy ślad domyka wszystkie trzy kroki. **TAK**.
- **possibly executable** $\mathbb{P}$ — natychmiast z powyższego. **TAK**.
- **necessary $\neg alive$ after** $\mathbb{P}$ — gałąź, w której *Spin* pozostawia broń rozładowaną, kończy się w σ_2 z *alive*. **NIE**.
- **possibly $\neg alive$ after** $\mathbb{P}$ — gałąź, w której *Spin* pozostawia broń naładowaną, kończy się w σ_3 . **TAK**.

4.2 Producent i konsumenci

Scenariusz oparty na przykładzie 5.8. W odróżnieniu od oryginału nie blokujemy sytuacji, w której obaj konsumenci pobierają produkt jednocześnie — w $\mathbb{DS4}$ nie można wyrazić **impossible** dla akcji złożonej. Daje to dodatkową dekompozycję i dodatkowy niedeterminizm.

Producent napełnia bufor, dwaj konsumenci A i B pobierają z niego produkt i zużywają go.

Sygnatura:

$$\begin{aligned}\mathcal{F} &= \{empty, hasA, hasB\}, \\ \mathcal{A} &= \{Put, GetA, GetB, ConsumeA, ConsumeB\}.\end{aligned}$$

Dziedzina D :

initially $\neg empty \wedge hasA \wedge hasB$	(bufor pełny, obaj konsumenci z produktem)
<i>Put</i> causes $\neg empty$	(producent napełnia bufor)
<i>GetA</i> causes $empty \wedge hasA$ if $\neg empty$	(A pobiera, o ile jest co pobrać)
<i>GetB</i> causes $empty \wedge hasB$ if $\neg empty$	(B pobiera, o ile jest co pobrać)
<i>ConsumeA</i> causes $\neg hasA$ if $hasA$	(A zużywa swój produkt)
<i>ConsumeB</i> causes $\neg hasB$ if $hasB$	(B zużywa swój produkt)

Brak **always** — wszystkie $2^3 = 8$ stanów są dopuszczalne. Stan początkowy: $\Sigma_0 = \{\sigma\}$, gdzie $\sigma = \{\neg empty, hasA, hasB\}$ (bufor pełny, obaj konsumenci mają produkt).

Akcja złożona $\mathbb{A} = \{Put, GetA, GetB, ConsumeA, ConsumeB\}$ w σ .

Konflikty w σ (oba warunki wstępne spełnione):

- *Put* vs. *GetA*: *Put* daje $\neg empty$, *GetA* daje $empty \wedge hasA$ — literały $\neg empty$ i $empty$ są komplementarne. **Konflikt** (Przypadek 1).
- *Put* vs. *GetB*: analogicznie. **Konflikt**.
- *GetA* vs. *ConsumeA*: *GetA* daje $empty \wedge hasA$, *ConsumeA* daje $\neg hasA$ — $hasA$ i $\neg hasA$ komplementarne. **Konflikt**.
- *GetB* vs. *ConsumeB*: analogicznie. **Konflikt**.
- *GetA* vs. *GetB*: oba dają $empty$ (ten sam znak) i różne fluenty $hasA, hasB$ — **brak konfliktu**. W oryginalnym przykładzie 5.8 *GetA* i *GetB* są wykluczone zdaniem **impossible** $\{GetA, GetB\}$. W $\mathbb{DS4}$ **impossible** dotyczy tylko akcji prostych, więc tej pary nie możemy tak zablokować.
- *ConsumeA* vs. *ConsumeB*, *Put* vs. *ConsumeA/ConsumeB*, *GetA* vs. *ConsumeB*, *GetB* vs. *ConsumeA*: brak konfliktu.

Dekompozycje — maksymalne podzbiory bez konfliktu:

- $\Delta_1 = \{Put, ConsumeA, ConsumeB\}$ — nie da się dołożyć $GetA$ ani $GetB$ (oba kolidują z Put).
- $\Delta_2 = \{GetA, GetB\}$ — nie da się dołożyć $Put, ConsumeA$ ani $ConsumeB$ (konflikty). *Ta dekompozycja nie istnieje w oryginalnym przykładzie 5.8.*
- $\Delta_3 = \{GetA, ConsumeB\}$ — nie da się dołożyć $Put, GetB$ ani $ConsumeA$ (konflikty).
- $\Delta_4 = \{GetB, ConsumeA\}$ — analogicznie.

Wyniki z $\sigma = \{\neg empty, hasA, hasB\}$:

- $\Delta_1: \neg empty \wedge \neg hasA \wedge \neg hasB \Rightarrow \sigma_1$.
- $\Delta_2: empty \wedge hasA \wedge hasB \Rightarrow \sigma_2$.
- $\Delta_3: empty \wedge hasA \wedge \neg hasB \Rightarrow \sigma_3$.
- $\Delta_4: empty \wedge \neg hasA \wedge hasB \Rightarrow \sigma_4$.

$Res(\mathbb{A}, \sigma) = \{\sigma_1, \sigma_2, \sigma_3, \sigma_4\}$. Oryginalne AC daje tylko $\{\sigma_1, \sigma_3, \sigma_4\}$ — brak σ_2 , bo tam $GetA$ i $GetB$ są razem niemożliwe.

Proces $\mathbb{P} = (\mathbb{A}, \{ConsumeA, ConsumeB\})$.

Krok 2 — $\{ConsumeA, ConsumeB\}$ w każdym stanie wynikowym (brak konfliktu, bo $\neg hasA$ i $\neg hasB$ dotyczą różnych fluentów):

- $\sigma_1 = \{\neg e, \neg A, \neg B\}$: oba warunki ($hasA, hasB$) niespełnione — efekt pusty, stan bez zmian.
- $\sigma_2 = \{e, A, B\}$: obaj konsumenci konsumują $\Rightarrow \sigma_0 = \{e, \neg A, \neg B\}$.
- $\sigma_3 = \{e, A, \neg B\}$: tylko A konsumuje $\Rightarrow \sigma_0$.
- $\sigma_4 = \{e, \neg A, B\}$: tylko B konsumuje $\Rightarrow \sigma_0$.

(Używamy skrótu $e = empty, A = hasA, B = hasB$.)

Po procesie $\mathbb{P}$: $Res = \{\sigma_0, \sigma_1\}$.

Kwerendy:

- **necessary executable** $\mathbb{P}$ — oba kroki mają $Res \neq \emptyset$. **TAK**.
- **possibly empty after** $\mathbb{P}$ — σ_0 spełnia $empty$. **TAK**.
- **necessary empty after** $\mathbb{P}$ — σ_1 spełnia $\neg empty$. **NIE**.
- **necessary $\neg hasA \wedge \neg hasB$ after** $\mathbb{P}$ — zarówno σ_0 , jak i σ_1 spełniają $\neg hasA \wedge \neg hasB$. **TAK** — niezależnie od przebiegu, po procesie żaden konsument nie ma produktu.

4.3 Dwa przełączniki, żarówka i alarm

Nowy przykład rozszerzający dwuprzełącznikowy scenariusz. Dwa przełączniki sw_1, sw_2 sterują żarówką (światło świeci, gdy włączony jest przynajmniej jeden z nich), a gdy świeci światło, uruchamia się alarm (czujnik ruchu). Dodatkowo akcja *On* wymusza zapalenie światła (np. ręczne zwarcie obwodu), a akcja *Flicker* modeluje zakłócenie zasilania — nie wymusza konkretnej wartości *light*, lecz zwalnia ten fluent spod inercji.

Scenariusz demonstruje: **always** (ograniczenia integralności i ramifikacje), **noninertial** (alarm zmienia się „za darmo” jako efekt uboczny), akcję złożoną bez konfliktu oraz konflikt Przypadek 2 (**causes** vs. **releases**).

Sygnatura:

$$\mathcal{F} = \{sw_1, sw_2, light, alarm\},$$

$$\mathcal{A} = \{Toggle_1, Toggle_2, On, Flicker\}.$$

Dziedzina D:

always $light \leftrightarrow (sw_1 \vee sw_2)$	(ramifikacja: światło świeci, gdy włączony choć jeden przełącznik)
always $light \rightarrow alarm$	(ramifikacja: świecąca żarówka aktywuje alarm)
noninertial $alarm$	(alarm nie podlega inercji — reaguje na światło)
$Toggle_1$ causes sw_1 if $\neg sw_1$	(przełącznik pierwszy: warunkowe causes)
$Toggle_1$ causes $\neg sw_1$ if sw_1	
$Toggle_2$ causes sw_2 if $\neg sw_2$	(przełącznik drugi: warunkowe causes)
$Toggle_2$ causes $\neg sw_2$ if sw_2	
On causes $light$	(wymuszenie zapalenia światła)
$Flicker$ releases $light$	(zakłócenie — światło może się zmienić)
initially $\neg sw_1 \wedge \neg sw_2$	(stan $alarm$ nieokreślony — opis częściowy)

Stany dopuszczalne Σ . Z $2^{|\mathcal{F}|} = 16$ wartościowań odrzucamy te, które łamią zdania **always**. Po pierwsze, musi zachodzić $light \leftrightarrow (sw_1 \vee sw_2)$, co wyklucza 8 stanów i wyznacza wartość $light$ na podstawie przełączników. Po drugie, gdy $light$ jest prawdziwe, musi być prawdziwy również $alarm$, co eliminuje jeszcze 3 stany. Pozostaje 5 stanów dopuszczalnych:

$$\begin{aligned}\sigma_0 &= \{\neg sw_1, \neg sw_2, \neg light, \neg alarm\}, \\ \sigma_1 &= \{\neg sw_1, \neg sw_2, \neg light, alarm\}, \\ \sigma_2 &= \{\neg sw_1, sw_2, light, alarm\}, \\ \sigma_3 &= \{sw_1, \neg sw_2, light, alarm\}, \\ \sigma_4 &= \{sw_1, sw_2, light, alarm\}.\end{aligned}$$

Zdanie **initially** wymusza $\neg sw_1, \neg sw_2$, natomiast nie ustala wartości fluentu $alarm$. Z dopuszczalnych stanów warunek ten spełniają zatem dwa: σ_0 oraz σ_1 . Ostatecznie $\Sigma_0 = \{\sigma_0, \sigma_1\}$ — jest to przypadek opisu częściowego (Z7).

Proces $\mathbb{P} = (\{Toggle_1, Toggle_2\}, \{On, Flicker\})$.

Krok 1: $\{Toggle_1, Toggle_2\}$ w σ_0 .

W stanie σ_0 aktywne są reguły $Toggle_i$ **causes** sw_i **if** $\neg sw_i$ (obu przełączników). Obie akcje są indywidualnie wykonalne.

Konflikty: efekty aktywne to sw_1 (DNF: $\{\{sw_1\}\}$) i sw_2 (DNF: $\{\{sw_2\}\}$). Literały dotyczą różnych fluentów, więc nie ma pary komplementarnej — brak konfliktu Przypadek 1. Żadna z akcji nie jest **releases**, więc brak Przypadek 2.

Dekompozycja: jedna, $\delta = \{Toggle_1, Toggle_2\}$.

Wynik: $E(\delta, \sigma_0) = \{sw_1, sw_2\}$, $R(\delta, \sigma_0) = \emptyset$.

$$\text{Res}_0(\delta, \sigma_0) = \{\sigma \in \Sigma : \sigma \models sw_1 \wedge sw_2\} = \{\sigma_4\}.$$

Jedyny kandydat, więc $\text{Res}(\{Toggle_1, Toggle_2\}, \sigma_0) = \{\sigma_4\}$ z $\text{New}(\sigma_0, \sigma_4) = \{sw_1, sw_2, light\}$. Zmiana $light$ jest wymuszona przez **always** (ramifikacja), zmiana $alarm$ jest „za darmo” — fluent nieinercyjny nie trafia do New.

Analogicznie dla drugiego stanu początkowego: w $\sigma_1 = \{\neg sw_1, \neg sw_2, \neg light, alarm\}$ aktywne reguły i zbiór $E(\delta, \sigma_1) = \{sw_1, sw_2\}$ są identyczne, więc $\text{Res}_0(\delta, \sigma_1) = \{\sigma_4\}$

i $\text{Res}(\{Toggle_1, Toggle_2\}, \sigma_1) = \{\sigma_4\}$. Obie gałęzie procesu zbiegają się zatem w σ_4 już po pierwszym kroku — nieokreśloność stanu początkowego alarmu została „wchłonięta” przez ramifikację $light \rightarrow alarm$ w nowym stanie.

Krok 2: $\{On, Flicker\}$ w σ_4 .

Indywidualna wykonalność:

- $\text{Res}(\{On\}, \sigma_4)$: $E = \{light\}$, $R = \emptyset$. $\text{Res}_0 = \{\sigma_2, \sigma_3, \sigma_4\}$ (wszystkie stany z $light$). Obliczenia New:

$$\text{New}(\sigma_4, \sigma_2) = \{\neg sw_1\}, \quad \text{New}(\sigma_4, \sigma_3) = \{\neg sw_2\}, \quad \text{New}(\sigma_4, \sigma_4) = \emptyset.$$

Minimum to $\emptyset$, zatem $\text{Res}(\{On\}, \sigma_4) = \{\sigma_4\}$ (akcja bez efektu — światło już świeci).

- $\text{Res}(\{Flicker\}, \sigma_4)$: $E = \emptyset$, $R = \{light\}$. $\text{Res}_0 = \Sigma = \{\sigma_0, \sigma_1, \sigma_2, \sigma_3, \sigma_4\}$. Fluent $light$ jest zwolniony — jego literal (zgodny z σ') wchodzi do New:

$$\begin{aligned} \text{New}(\sigma_4, \sigma_0) &= \{\neg sw_1, \neg sw_2, \neg light\}, \\ \text{New}(\sigma_4, \sigma_1) &= \{\neg sw_1, \neg sw_2, \neg light\}, \\ \text{New}(\sigma_4, \sigma_2) &= \{\neg sw_1, light\}, \\ \text{New}(\sigma_4, \sigma_3) &= \{\neg sw_2, light\}, \\ \text{New}(\sigma_4, \sigma_4) &= \{light\}. \end{aligned}$$

Minimum względem inkluzji to $\{light\}$ (dla σ_4) oraz $\{\neg sw_1, \neg sw_2, \neg light\}$ (dla σ_0 i σ_1 , nieporównywalne z $\{light\}$). Zatem $\text{Res}(\{Flicker\}, \sigma_4) = \{\sigma_0, \sigma_1, \sigma_4\}$.

Konflikty: $E(On, \sigma_4) = \{light\}$, $\text{DNF}(light) = \{\{light\}\}$; $Flicker$ zwalnia fluent $light$, który występuje w tym składniku — to **konflikt Przypadek 2 (causes vs. releases)**.

Dekompozycje: graf konfliktu na $V_{\text{exec}} = \{On, Flicker\}$ ma jedną krawędź $\{On, Flicker\}$. Maksymalne zbiory bezkonfliktowe to dwa singletony: $\delta_1 = \{On\}$, $\delta_2 = \{Flicker\}$.

Wynik:

$$\text{Res}(\{On, Flicker\}, \sigma_4) = \text{Res}(\delta_1, \sigma_4) \cup \text{Res}(\delta_2, \sigma_4) = \{\sigma_4\} \cup \{\sigma_0, \sigma_1, \sigma_4\} = \{\sigma_0, \sigma_1, \sigma_4\}.$$

Stany osiągalne po procesie $\mathbb{P}$: $\{\sigma_0, \sigma_1, \sigma_4\}$.

Kwerendy:

- **necessary executable $\mathbb{P}$** — Z obu stanów początkowych σ_0, σ_1 krok 1 prowadzi do σ_4 , a krok 2 ma w σ_4 niepuste Res, więc każda ścieżka dochodzi do poziomu 2. **TAK**.
- **possibly executable $\mathbb{P}$** — wynika z **necessary executable**: z każdego stanu początkowego istnieje (wręcz każdy) ślad domykający proces. **TAK**.
- **necessary alarm after $\mathbb{P}$** — σ_0 spełnia $\neg alarm$, więc istnieje ścieżka kończąca się bez alarmu. **NIE**.
- **possibly alarm after $\mathbb{P}$** — z każdego stanu początkowego (σ_0, σ_1) istnieje ślad kończący się w $\sigma_4 \models alarm$. **TAK**.

Komentarz. W kroku 1 **causes** wymusza jedynie sw_1, sw_2 ; zmiana $light$ wynika z **always** (ramifikacja) i wchodzi do New, a $alarm$ zmienia się „za darmo” dzięki **noninertial**. W kroku 2 $Flicker$ (**releases**) daje kilka minimalnych stanów, a w parze z On tworzy konflikt Przypadku 2 rozbijający akcję złożoną na dwie dekompozycje.

4.4 Dwóch robotników wynoszących stół

Autorski scenariusz integrujący wszystkie mechanizmy dziedziny $\mathbb{DS4}$: opis częściowy (Z7), ograniczenia integralności, fluenty nieinercyjne, ramifikacje, warunkowe **causes** i **releases**, **impossible** aktywowaną dynamicznie, obie kategorie konfliktu akcji złożonej (Przypadek 1 i Przypadek 2) oraz zdanie obserwacyjne przycinające zbiór Σ_0 . Na jednej dziedzinie porównamy dwa procesy $\mathbb{P}_1$ i $\mathbb{P}_2$, tak by w sumie trafić wszystkie interesujące ścieżki.

Dwóch robotników, A i B , ma wynieść stół z pokoju. Każdy może samodzielnie go podnosić lub puszczać ($LiftA/B$, $DropA/B$), otwierać lub zamykać drzwi ($OpenA/B$, $CloseA/B$) oraz wspólnie wynieść stół na zewnątrz ($CarryOut$). Stół uznajemy za uniesiony tylko wtedy, gdy trzymają go jednocześnie obaj robotnicy. Drzwi można obsługiwać wyłącznie wolną ręką.

Sygnatura:

$$\mathcal{F} = \{holdA, holdB, tableUp, doorOpen, broken, outside\},$$

$$\mathcal{A} = \{LiftA, LiftB, DropA, DropB, OpenA, OpenB, CloseA, CloseB, CarryOut\}.$$

Dla zwięzłości poniżej używamy skrótów $a = holdA$, $b = holdB$, $u = tableUp$, $d = doorOpen$, $r = broken$, $o = outside$ i zapisujemy stan jako szóstkę (a, b, u, d, r, o) .

Dziedzina D :

Ramifikacje i fluenty nieinercyjne:

always $u \leftrightarrow (a \wedge b)$	(stół uniesiony $\Leftrightarrow$ trzymają go obaj)
noninertial u	(stół zależy od robotników)

Podnoszenie i opuszczanie stołu:

$LiftA$ causes a , $LiftB$ causes b	(pojedynczy robotnik chwyta stół)
$DropA$ causes $\neg a$, $DropB$ causes $\neg b$	(pojedynczy robotnik puszcza stół)

Obsługa drzwi (tylko wolną ręką):

$OpenA$ causes d , $OpenB$ causes d	(dowolny robotnik otwiera drzwi)
$CloseA$ causes $\neg d$, $CloseB$ causes $\neg d$	(dowolny robotnik zamyka drzwi)
impossible $OpenA$ if a , impossible $OpenB$ if b	(potrzeba wolnej ręki)
impossible $CloseA$ if a , impossible $CloseB$ if b	(analogicznie dla zamykania)

Wynoszenie stołu:

$CarryOut$ causes o if d	(wyjście tylko przez otwarte drzwi)
impossible $CarryOut$ if $\neg u$	(nie można wynieść leżącego stołu)
$CarryOut$ releases a if $\neg d$	
$CarryOut$ releases b if $\neg d$	
$CarryOut$ releases r if $\neg d$	(uderzenie w drzwi — chaos)

Opis stanu początkowego i obserwacje:

initially $\neg a \wedge \neg b \wedge \neg o$	(nie trzymają, stół w środku; d, r nieustalone)
observable $\neg r$ after $LiftA, LiftB, CarryOut$	(obserwacja — da się wynieść w całości)

Stany dopuszczalne Σ . Spośród $2^6 = 64$ wartościowań ograniczenie **always** $u \leftrightarrow (a \wedge b)$ wyznacza u na podstawie a, b , co daje $|\Sigma| = 2^5 = 32$ stany (swobodne a, b, d, r, o).

Zbiór Σ_0 — krok 1: zdanie **initially.** Warunek $\neg a \wedge \neg b \wedge \neg o$ ustala trzy fluenty; u jest wyliczone ($u = F$ dla $a = b = F$). Swobodne pozostają d, r , co daje cztery kandydatury:

$$\begin{aligned} s_1 &= (F, F, F, F, F, F), & s_2 &= (F, F, F, F, T, F), \\ s_3 &= (F, F, F, T, F, F), & s_4 &= (F, F, F, T, T, F). \end{aligned}$$

Zbiór Σ_0 — krok 2: przycięcie zdaniem obserwacyjnym. Zdanie **observable** $\neg r$ **after** $LiftA, LiftB, CarryOut$ wymaga, by *istniała* ścieżka wykonania sekwencji w której stan końcowy spełnia $\neg r$. W każdym kandydacie s_i po $LiftA, LiftB$ mamy $a = b = u = T$, wartości d, r niezmienione. W kroku $CarryOut$:

- gdy $d = F$ (stany s_1, s_2): $\neg d$ aktywuje **releases** dla a, b, r , natomiast **causes** o **if** d jest nieaktywne. Fluent r jest zwolniony, więc istnieje stan wynikowy z $r = F$ — warunek obserwacyjny spełniony.
- gdy $d = T$ (stany s_3, s_4): **releases** nieaktywne, r pozostaje bez zmian przez inercję. s_3 ma $r = F$ i warunek spełnia; s_4 ma $r = T$, a jedyny stan wynikowy też ma $r = T$ — warunek zawodzi.

Odrzucamy więc wyłącznie s_4 . Ostatecznie

$$\Sigma_0 = \{\sigma_0, \sigma_1, \sigma_2\}, \quad \sigma_0 = s_1, \quad \sigma_1 = s_2, \quad \sigma_2 = s_3.$$

Jest to przypadek opisu częściowego (Z7): trzy stany początkowe, różniące się wartością d (drzwi) i r (stan stołu) poza zakresem **initially**.

Poniżej rozważymy dwa procesy, które razem pokazują wszystkie mechanizmy semantyki. Każdy z nich używamy jako $\mathbb{P}$ w osobnym komplecie kwerend.

Proces $\mathbb{P}_1 = (\{OpenA\}, \{LiftA, DropA\}, \{LiftB\}, \{CarryOut\})$. Cel: pokazać (i) akcję singletonową z warunkową **causes**, (ii) konflikt Przypadek 1 w kroku 2, (iii) dynamiczne **impossible** aktywujące się dopiero na końcu procesu.

Krok 1: $\{OpenA\}$. Singleton — brak konfliktu. W każdym $\sigma \in \Sigma_0$ mamy $a = F$, więc **impossible** $OpenA$ **if** a jest nieaktywne.

- $\sigma_0 \rightarrow \sigma_2$ (jedyna zmiana: $d: F \rightarrow T$).
- $\sigma_1 \rightarrow \sigma_3 := (F, F, F, T, T, F)$.
- $\sigma_2 \rightarrow \sigma_2$ (d już prawdziwe — efekt pusty, $New = \emptyset$).

Zbiór osiągalny: $\{\sigma_2, \sigma_3\}$.

Krok 2: $\{LiftA, DropA\}$. W obu stanach obie akcje są indywidualnie wykonalne. $E(LiftA) = \{a\}$, $DNF = \{\{a\}\}$; $E(DropA) = \{\neg a\}$, $DNF = \{\{\neg a\}\}$. Literały a i $\neg a$ są komplementarne — **konflikt Przypadek 1**. Dekompozycje: $\delta_1 = \{LiftA\}$, $\delta_2 = \{DropA\}$.

Wyniki (stan zapisujemy przed $\rightarrow$ po):

- z σ_2 : δ_1 daje $\sigma_4 := (T, F, F, T, F, F)$; δ_2 daje σ_2 (efekt $\neg a$ już spełniony). $Res = \{\sigma_2, \sigma_4\}$.
- z σ_3 : analogicznie $\sigma_5 := (T, F, F, T, T, F)$ oraz σ_3 . $Res = \{\sigma_3, \sigma_5\}$.

Krok 3: $\{LiftB\}$. Singleton — brak konfliktu, brak **impossible**. Akcja wymusza b ; ramifikacja **always** dopasowuje u do $a \wedge b$ (tutaj u nieinercyjny, więc nie trafia do New):

- $\sigma_2 \rightarrow \sigma_6 := (F, T, F, T, F, F)$ (u pozostaje F , bo $a = F$).
- $\sigma_3 \rightarrow \sigma_7 := (F, T, F, T, T, F)$.
- $\sigma_4 \rightarrow \sigma_8 := (T, T, T, T, F, F)$ (u wchodzi w T przez ramifikację).
- $\sigma_5 \rightarrow \sigma_9 := (T, T, T, T, T, F)$.

Krok 4: $\{CarryOut\}$. Singleton, ale **impossible** $CarryOut$ **if** $\neg u$ blokuje akcję w stanach z $u = F$:

- σ_6, σ_7 : $u = F$ — **niewykonalne**. Ślad Ψ na tej gałęzi pozostaje niezdefiniowany.
- σ_8 : $u = T$, $d = T$, więc **causes** o **if** d wymusza o , **releases** (**if** $\neg d$) nieaktywne. $Res = \{\sigma_{10}\}$, $\sigma_{10} := (T, T, T, T, F, T)$.

- σ_9 : analogicznie, $\text{Res} = \{\sigma_{11}\}$, $\sigma_{11} := (\text{T}, \text{T}, \text{T}, \text{T}, \text{T}, \text{T})$.

Drzewo śladów $\mathbb{P}_1$:

$$\begin{array}{l}
\sigma_0 \rightarrow \sigma_2 \rightarrow \sigma_2 \rightarrow \sigma_6 \rightarrow \text{niewykonalne} \\
\sigma_4 \rightarrow \sigma_8 \rightarrow \sigma_{10} \text{ (SUCCESS)} \\
\sigma_1 \rightarrow \sigma_3 \rightarrow \sigma_3 \rightarrow \sigma_7 \rightarrow \text{niewykonalne} \\
\sigma_5 \rightarrow \sigma_9 \rightarrow \sigma_{11} \text{ (SUCCESS)} \\
\sigma_2 \rightarrow \sigma_2 \rightarrow \sigma_2 \rightarrow \sigma_6 \rightarrow \text{niewykonalne} \\
\sigma_4 \rightarrow \sigma_8 \rightarrow \sigma_{10} \text{ (SUCCESS)}
\end{array}$$

Kwerendy dla $\mathbb{P}_1$:

- **necessary executable $\mathbb{P}_1$** — z każdego $\sigma \in \Sigma_0$ istnieje ślad kończący się „niewykonalne” (gałąź przez $\delta_2 = \{\text{DropA}\}$). **NIE**.
- **possibly executable $\mathbb{P}_1$** — z każdego $\sigma \in \Sigma_0$ istnieje ślad pełny (gałąź przez $\delta_1 = \{\text{LiftA}\}$). **TAK**.
- **necessary o after $\mathbb{P}_1$** — na gałęziach niewykonalnych Ψ jest niezdefiniowane, więc kwerenda konieczności nie jest spełniona. **NIE**.
- **possibly o after $\mathbb{P}_1$** — z każdego stanu początkowego gałąź sukcesu kończy się w σ_{10} lub σ_{11} , spełniających o . **TAK**.

Proces $\mathbb{P}_2 = (\{\text{LiftA}, \text{LiftB}\}, \{\text{LiftA}, \text{CarryOut}\})$. Cel: pokazać (i) akcję złożoną bez konfliktu w kroku 1, (ii) konflikt Przypadek 2 (**causes** vs. aktywne **releases**) w kroku 2, (iii) warunkową **releases** rozbijającą świat na wiele stanów wynikowych.

Krok 1: $\{\text{LiftA}, \text{LiftB}\}$. $E(\text{LiftA}) = \{a\}$, $E(\text{LiftB}) = \{b\}$ — różne fluenty, żadna akcja nie jest **releases**, więc **brak konfliktu** (ani Przypadek 1, ani 2). Jedna dekompozycja $\delta = \{\text{LiftA}, \text{LiftB}\}$, $E(\delta) = \{a, b\}$, $R(\delta) = \emptyset$.

Z każdego stanu $\sigma \in \Sigma_0$ otrzymujemy jeden stan wynikowy (ramifikacja wymusza $u = \text{T}$, fluent nieinercyjny):

- $\sigma_0 \rightarrow \sigma_{12} := (\text{T}, \text{T}, \text{T}, \text{F}, \text{F}, \text{F})$.
- $\sigma_1 \rightarrow \sigma_{13} := (\text{T}, \text{T}, \text{T}, \text{F}, \text{T}, \text{F})$.
- $\sigma_2 \rightarrow \sigma_8$ (ten sam σ_8 co w $\mathbb{P}_1$).

Krok 2: $\{\text{LiftA}, \text{CarryOut}\}$. Tu dzieje się najwięcej. Analiza zależy od tego, czy d jest prawdziwe w stanie, w którym wykonujemy akcję złożoną.

W stanach σ_{12}, σ_{13} ($d = \text{F}$). *CarryOut*: $u = \text{T}$, akcja wykonalna; $\neg d = \text{T}$, więc *aktywna* jest **releases** dla a, b, r , natomiast **causes o if** d jest *nieaktywne*. Zatem $E(\text{CarryOut}) = \emptyset$, $R(\text{CarryOut}) = \{a, b, r\}$. *LiftA*: $E = \{a\}$, $\text{DNF} = \{\{a\}\}$ — fluent a występuje w DNF zarówno w efekcie **causes LiftA**, jak i w aktywnej **releases CarryOut**. **Konflikt Przypadek 2 (causes vs. releases)**.

Dekompozycje: $\delta_1 = \{\text{LiftA}\}$, $\delta_2 = \{\text{CarryOut}\}$.

- δ_1 w σ_{12} : a już prawdziwe, $\text{New} = \emptyset$, $\text{Res}(\delta_1, \sigma_{12}) = \{\sigma_{12}\}$; analogicznie $\text{Res}(\delta_1, \sigma_{13}) = \{\sigma_{13}\}$.
- δ_2 w σ_{12} : $E = \emptyset$, $R = \{a, b, r\}$. Fluenty niezwolnione (d, o) muszą pozostać, czyli $d = o = \text{F}$; fluenty zwolnione (a, b, r) mogą się zmieniać swobodnie; u wyliczane przez ramifikację. Dla

każdej z $2^3 = 8$ kombinacji (a, b, r) mamy $\text{New} = \emptyset$ (żaden z fluentów inercyjnych nie wymusza zmiany). Wszystkie 8 kandydatów jest minimalnych:

$$\begin{aligned}\tau_1 &= (T, T, T, F, T, F) = \sigma_{13}, & \tau_2 &= (T, T, T, F, F, F) = \sigma_{12}, \\ \tau_3 &= (T, F, F, F, T, F), & \tau_4 &= (T, F, F, F, F, F), \\ \tau_5 &= (F, T, F, F, T, F), & \tau_6 &= (F, T, F, F, F, F), \\ \tau_7 &= (F, F, F, F, T, F) = \sigma_1, & \tau_8 &= (F, F, F, F, F, F) = \sigma_0.\end{aligned}$$

$\text{Res}(\delta_2, \sigma_{12}) = \{\tau_1, \dots, \tau_8\}$; tak samo $\text{Res}(\delta_2, \sigma_{13}) = \{\tau_1, \dots, \tau_8\}$ (zbiór zwolnionych fluentów pokrywa wszystkie fluenty, którymi oba stany się różnią).

$\text{Res}(\{LiftA, CarryOut\}, \sigma_{12}) = \text{Res}(\delta_1, \sigma_{12}) \cup \text{Res}(\delta_2, \sigma_{12}) = \{\tau_1, \dots, \tau_8\}$ (8 stanów, wszystkie z $o = F$). Analogicznie dla σ_{13} .

W stanie σ_8 ($d = T$). Teraz $\neg d = F$, więc wszystkie trzy **releases** są *nieaktywne*; **causes o if d aktywne**. $E(CarryOut) = \{o\}$, $R(CarryOut) = \emptyset$. Z *LiftA* brak wspólnych literałów komplementarnych ani zwolnionych fluentów z DNF — **brak konfliktu**. Jedna dekompozycja $\delta = \{LiftA, CarryOut\}$, $E(\delta) = \{a, o\}$. Stan σ_8 już spełnia a ; jedyny minimalny stan wynikowy to σ_{10} (z $\mathbb{P}_1$). Zatem $\text{Res}(\{LiftA, CarryOut\}, \sigma_8) = \{\sigma_{10}\}$.

Drzewo śladów $\mathbb{P}_2$:

$$\begin{aligned}\sigma_0 &\rightarrow \sigma_{12} \rightarrow \{\tau_1, \tau_2, \tau_3, \tau_4, \tau_5, \tau_6, \tau_7, \tau_8\} \quad (\text{wszystkie mają } o = F) \\ \sigma_1 &\rightarrow \sigma_{13} \rightarrow \{\tau_1, \tau_2, \tau_3, \tau_4, \tau_5, \tau_6, \tau_7, \tau_8\} \\ \sigma_2 &\rightarrow \sigma_8 \rightarrow \{\sigma_{10}\} \quad (\text{ma } o = T)\end{aligned}$$

Każdy ślad Ψ jest zdefiniowany — proces $\mathbb{P}_2$ jest wykonalny w każdym przypadku.

Kwerendy dla $\mathbb{P}_2$:

- **necessary executable $\mathbb{P}_2$** — w żadnym węźle drzewa wykonania nie zachodzi blokada. **TAK**.
- **possibly executable $\mathbb{P}_2$** — natychmiast z powyższego. **TAK**.
- **necessary o after $\mathbb{P}_2$** — gałąź startująca z σ_0 kończy się w zbiorze $\{\tau_1, \dots, \tau_8\}$, w którym wszędzie $o = F$. Decyduje o tym wybór dekompozycji $\delta_2 = \{CarryOut\}$ w kroku 2 — bo $d = F$, **causes** dla o nie odpala, a **releases** nie wymusza o . **NIE**.
- **possibly o after $\mathbb{P}_2$** — semantyka **possibly** wymaga, by z *każdego* stanu początkowego istniała ścieżka kończąca się w stanie spełniającym o . Ze stanu σ_2 (drzwi otwarte) rzeczywiście $\sigma_2 \rightarrow \sigma_8 \rightarrow \sigma_{10} \models o$, jednak ze stanów σ_0, σ_1 (drzwi zamknięte) proces $\mathbb{P}_2$ nigdy nie otwiera drzwi: **causes o if d** pozostaje nieaktywne, a wszystkie stany wynikowe $\tau_1, \dots, \tau_8$ mają $o = F$. Skoro dla σ_0, σ_1 nie istnieje ścieżka osiągająca o , kwerenda jest fałszywa. **NIE**.

Komplementarność procesów. $\mathbb{P}_1$ pokazuje konflikt Przypadku 1 i dynamiczną niewykonalność *CarryOut* (stąd **necessary executable** fałszywe), a $\mathbb{P}_2$ — konflikt Przypadku 2 z niedeterministyczną **releases**. $\mathbb{P}_2$ ilustruje też uniwersalny kwantyfikator po Σ_0 : **possibly o after $\mathbb{P}_2$** jest fałszywa, bo ze stanów początkowych z zamkniętymi drzwiami cel o jest nieosiągalny, mimo że ze stanu σ_2 — osiągalny.

5 Podsumowanie

W niniejszym opracowaniu zaprojektowano kompletny formalny aparat do opisu oraz analizy systemów dynamicznych klasy DS4. W szczególności zdefiniowano:

- **język akcji**, umożliwiający formalny opis dziedziny w oparciu o akcje proste, ich efekty, warunki wykonalności oraz ograniczenia integralności,
- **semantykę operacyjną**, obejmującą zarówno pojedyncze akcje, jak i akcje złożone, wraz z mechanizmem konfliktów, dekompozycji oraz minimalizacji zmian,
- **model dziedziny**, który jednoznacznie określa zbiór stanów dopuszczalnych, stanów początkowych oraz funkcję przejścia,
- **język kwerend**, pozwalający formułować pytania dotyczące wykonalności procesów oraz osiągalności określonych stanów końcowych.

Kluczowe było scalenie mechanizmów języka *AR* (inercja, ramifikacje, niedeterminizm) oraz *AC* (konflikty i dekompozycje akcji złożonych) w jednej semantyce, wraz z uogólnieniem warunku rozłączności zmiennych z treści zadania. Istotnym rozszerzeniem względem klasycznego *AR* jest dopuszczenie wielu stanów początkowych (Σ_0 , założenie *Z7*), uwzględniane przy interpretacji kwerend *necessary/possibly*.